

American Options Pricing Via CTMC Approximation

Content List

Member List

Preliminary Knowledge

Approximation Theorem

- Basic concepts
- Generator Calculation
 - Diffusion process
 - Jump Diffusion process
- Matrix Q of the constructed CTMC
 - Setup for the state space X
 - Example
- Diffusion process
 - Construction Process
 - Example of finding drift and diffusion
- Jump Diffusion Process
 - Construction Process
- Regime - Switching Jump Diffusion Process
 - Construction Process
 - Step 1: Intra-Regime Diffusion Blocks
 - Step 2: Inter-Regime Switching Blocks
 - Step 3: Diagonal Block Correction

Parameter Estimation

Pricing Process

Examples

- Example 1
 - Model Specification
 - Approximation Process
 - Parameter Estimation
 - Pricing Process
- Example 2
 - Model Specification
 - Approximation Process
 - Step 1: State Space Discretization
 - Step 2: Constructing the Generator Matrix Q
 - Part A: Constructing the Jump Component Q^J
 - Part B: Constructing the Diffusion Component Q^D
 - Part C: Assembling and Validating the Complete Matrix Q
 - Parameter Estimation
 - Likelihood Function Construction
 - Log-Likelihood and Optimization
 - Special Considerations for CEV-DEJD
 - Validation of Estimates
 - Pricing Process
 - Backward Induction Algorithm
 - Special Considerations for CEV-DEJD
 - Final Price Computation
- Example 3

- Model Specification
- Approximation Process
 - State Space Setup
 - Constructing the Generator Matrix Q
- Parameter Estimation
 - State-Dependent Observation Density
 - Hamilton Filter Recursion
 - Complete Log-Likelihood Function
- Pricing Process
 - Time Discretization and Bermudan Approximation
 - State Value Matrix
 - Backward Induction
 - Final Pricing
 - State-Dependent Optimal Exercise Boundaries

Approximation Error

- Motivation
- Error Definition
- Detailed Analysis
- Plain Language

Appendix

- Appendix A: Derivation of the Matrix Exponential Formula $P_t = e^{tQ}$
- Appendix B: Derivation of the Generator of diffusion process via Moment Matching
- Appendix C: Understanding Jump Terms in Stochastic Processes
- Appendix D: Derivation of the Generator of Jump Diffusion process via Moment Matching

Member List

Yi Hong (FAM/SMP)

Shunyi Wang (FM/2144277)

Shuhan Lin (FM/2143174)

Junyi Zhou (FM/2252703)

Preliminary Knowledge

A **stochastic process** is a group of random variables indexed by time. Formally, a stochastic process $\{X_t, t \in T\}$ is a family of random variables defined on a probability space, where:

- t is the time parameter
- T is the time index set (can be a discrete set or a continuous interval)
- X_t represents the random state at time t

Plain Language: A stochastic process is simply a way to describe how something random changes over time. Imagine you're watching a dice being rolled every hour - you can't predict what number will come up next, but you know it will be between 1 and 6. The sequence of all these dice rolls over time is a stochastic process.

Example: Think about checking your phone for text messages throughout the day. At any given hour, the number of new messages you receive is random - sometimes you get none, sometimes 5, sometimes 1. The sequence showing "messages received each hour" over an entire day forms a stochastic process. Each hour represents a time point, and the number of messages at that time is the random variable.

A **Markov chain** is a stochastic process $\{X_t, t \in T\}$ where the probability of moving to the next state depends only on the current state and not on the sequence of events that preceded it (This is called Markov property).

Mathematically, we can express the Markov property as follows:

$$\mathbb{P}(X_t = j \mid \mathcal{F}_s) = \mathbb{P}(X_t = j \mid X_s)$$

Plain Language: A Markov chain is like a person with very short memory - it only cares about "where am I **now**?" and completely forgets the past journey "how did I get here?". The next step depends entirely on your current location, not the travel history.

Example: Imagine you're playing a board game where you move based on dice rolls. At each turn, your next position depends only on where you currently are and the dice roll - it doesn't matter whether you got to your current position by taking the long route or the shortcut.

The Markov chain can be divided into two categories: the discrete one and the continuous one, which is determined by the index set T . If this is a discrete set, this chain will be discrete. If the set is a continuous interval, this chain will be continuous.

In the discrete Markov chain, the critical determinant is the **state transition matrix**, which gives the principle for the transitions between different states.

For a discrete-time Markov chain with finite state space $S = \{1, 2, \dots, n\}$, the **transition probability matrix** $\mathbf{P}$ is defined as:

$$P_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i)$$

where P_{ij} represents the probability of transitioning from state i to state j in one time step. The matrix $\mathbf{P}$ satisfies:

- $P_{ij} \geq 0$ for all i, j (non-negative entries)
- $\sum_{j=1}^n P_{ij} = 1$ for all i (each row sums to 1)

Plain Language: The P -matrix is like a "jump rule table" - it tells you the probability of jumping from each state to every other state. Imagine you're playing a hopscotch game where the P -matrix is your rulebook. The number in row i and column j tells you: "If you're currently in square i , what's the probability of jumping to square j in your next move?"

Example: Consider a simple weather model with only two states: Sunny (S) and Rainy (R). Suppose:

- If today is sunny, tomorrow will be sunny with probability 0.7, and rainy with probability 0.3
- If today is rainy, tomorrow will be sunny with probability 0.4, and rainy with probability 0.6

Then the transition probability matrix is:

Unknown environment 'pmatrix*'

where the first row represents transition probabilities starting from sunny weather, and the second row represents transitions starting from rainy weather.

For continuous-time Markov chains, we focus on the **generator matrix** $\mathbf{Q}$ instead of the transition probability matrix. Readers might be curious about the reason for this change of the research perspective. In simple terms, The generator matrix $\mathbf{Q}$ is defined through the **transition rate** from state i to state j and satisfy the following properties:

1. The sum of each row is equal to 0. (mathematical formula)
2. The diagonal elements are non-positive and the off diagonal elements are nonnegative.

Therefore, the transition rate matrix of CTMC is like:

$$\mathbf{Q} = \begin{pmatrix} -q_1 & q_{12} & \cdots & q_{1n} \\ q_{21} & -q_2 & \cdots & q_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ q_{n1} & q_{n2} & \cdots & -q_n \end{pmatrix}$$

Plain Language: The Q-matrix is like a "transition speed table". Unlike the P-matrix which directly tells you "jumping probabilities", the Q-matrix tells you the information about "jumping rates" or "transition intensities". Imagine you're moving between different rooms. The element q_{ij} tells you the intensity of your urge to rush from room i to room j ($i \neq j$). The diagonal entries (usually denoted as q_{ii}) are negative numbers representing your "total urge to escape from the current room" - think of it as an "escape rate".

Example: Consider a server status model with three states: Normal Operation (1), Warning (2), and Failure (3). Suppose that we have the following known conditions,

1. Rate from Normal to Warning: $q_{12} = 0.1$
2. Rate from Normal to Failure: $q_{13} = 0.02$
3. Rate from Warning to Normal: $q_{21} = 0.3$
4. Rate from Warning to Failure: $q_{23} = 0.05$
5. Rate from Failure to Normal: $q_{31} = 0.2$
6. Rate from Failure to Warning: $q_{32} = 0.1$

Using the constraint that each row sums to zero, we can calculate other elements in the generator

- $q_{11} = -(q_{12} + q_{13}) = -(0.1 + 0.02) = -0.12$
- $q_{22} = -(q_{21} + q_{23}) = -(0.3 + 0.05) = -0.35$
- $q_{33} = -(q_{31} + q_{32}) = -(0.2 + 0.1) = -0.3$

Therefore, the generator matrix is:

$$\mathbf{Q} = \begin{pmatrix} -0.12 & 0.1 & 0.02 \\ 0.3 & -0.35 & 0.05 \\ 0.2 & 0.1 & -0.3 \end{pmatrix}$$

In this project, the relationship between the generator and the transition matrix of CTMC is as follows:

$$P_t = e^{tQ}$$

The proof of this important principle is given in the Appendix A.

A **Birth-Death process** is a continuous-time Markov chain where the system can only move to neighboring states (± 1). It has:

- Birth rate λ_i : transitions from state i to $i+1$
- Death rate μ_i : transitions from state i to $i-1$

Plain Language: This is like counting people in an elevator - the number can only change by one person at a time unit. Someone gets on (+1) or someone gets off (-1), but you can't have 5 people suddenly appear or disappear at once.

Example: Consider a single-threaded server processing a task queue. Tasks arrive one at a time to be processed, and the server can only complete one task at a time. If there are currently 3 tasks in the queue (including the one being processed), the next change can only be either 4 tasks (a new task arrives) or 2 tasks (the current task is completed). The server cannot simultaneously start processing multiple tasks or finish several tasks at once due to its single-threaded nature. The arrival rate might vary based on system load, and processing speed might depend on task complexity.

A **diffusion process** is a type of **continuous-time Markov process** with **continuous sample paths**, typically used to model systems that evolve smoothly but randomly over time.

It satisfies the following **local characteristics**:

1. Drift: $\lim_{\Delta t \rightarrow 0} \frac{1}{\Delta t} \mathbb{E}[\Delta X_t | X_t = x] = \mu(x, t)$
2. Diffusion: $\lim_{\Delta t \rightarrow 0} \frac{1}{\Delta t} \mathbb{E}[(\Delta X_t)^2 | X_t = x] = \sigma^2(x, t)$
3. Jump absence: $\lim_{\Delta t \rightarrow 0} \frac{1}{\Delta t} \mathbb{P}(|\Delta X_t| > \epsilon | X_t = x) = 0 \quad \forall \epsilon > 0$

Plain Language: Imagine tracking a balloon in the wind. The wind gives it overall direction (drift). But air currents make it wiggle randomly (diffusion). Moreover, this balloon moves smoothly without the sudden jump.

Example: Consider the temperature of a room with air conditioning. The temperature has a general trend toward the thermostat setting (drift), but it fluctuates randomly due to factors like people opening doors, sunlight through windows, or the AC cycling on and off (diffusion). The temperature changes continuously and smoothly - it cannot jump instantly from 72°F to 68°F, but must pass through all intermediate values like 71.8°F, 71.5°F, etc. (continuous sample paths with no jumps).

Mathematically, **diffusion processes are often defined as the solution to a stochastic differential equation (SDE)**. That is, if X_t satisfies the following SDE:

$$dX_t = \mu(X_t, t) dt + \sigma(X_t, t) dW_t$$

then under suitable conditions, X_t is a diffusion process.

- W_t is a Wiener process (also called the standard Brownian motion)
- $\mu(X_t, t)$ is the drift coefficient
- $\sigma(X_t, t)$ is the diffusion coefficient

This SDE gives a precise way to **construct** a diffusion process: it prescribes how the system evolves both deterministically (via μ) and stochastically (via σ).

Plain Language: A SDE is a differential equation that describes how something changes over time, but includes both predictable changes and random fluctuations. It's like having a "recipe" that tells you how a system evolves, where the recipe includes both deterministic rules and random ingredients.

Example: Consider modeling how a stock price changes. An SDE gives us the rule for price movement:

- The equation: $dS_t = \mu S_t dt + \sigma S_t dW_t$
- $\mu S_t dt$: the expected return component (if $\mu = 0.05$, we expect 5% annual growth)
- $\sigma S_t dW_t$: the random volatility component (market noise, unexpected news)
- The SDE above is the easiest **mathematical rule** that generates the random stock price path we observe

An **option** is a contract which conveys to its owner, the *holder*, the right, but not the obligation, to buy or sell a specific quantity of an underlying asset or instrument at a specified strike price on or before a specified date, depending on the style of the option.

The core elements of an option are as follows:

- **Underlying assets**: stocks, indexes, and futures
- **Strike price**: the transaction price predetermined in the contract by both sides of the transaction
- **Time to maturity**: the validity period of the option
- **Premium**: the cost of buying this option

Options can be **classified** by the following ways:

- **According to the option rights:**
 - Call options: give the buyer of the option the right to buy the underlying assets as stipulated in the contract
 - Put options: give the buyer of the option the right to sell the underlying assets as stipulated in the contract
- **According to the option styles**
 - American option** – an option that may be exercised on any trading day on or before expiration
 - European option** – an option that may only be exercised on expiry
 - Bermudan option** – an option that may be exercised only on specified dates on or before expiration
 - Asian option** – an option whose payoff is determined by the average underlying price over some preset time period
 - Barrier option** – any option with the general characteristic that the underlying security's price must pass a certain level or "barrier" before it can be exercised
 - Binary option** – An all-or-nothing option that pays the full amount if the underlying security meets the defined condition on expiration, otherwise, it expires.
 - Exotic option** – any of a broad category of options that may include complex financial structures

The value of an option is composed of as follows:

option value = intrinsic value + time value

Intrinsic value: when an option contract is immediately exercised, the profit that the buyer can obtain

At time t , for the call option, the intrinsic value is $\max(0, S_t - K)$; for the put option, the intrinsic value is $\max(0, K - S_t)$.

According to the intrinsic value, we can get another classification of options:

- In-the-money option/ ITM (intrinsic value > 0)
- At-the-money option/ ATM (intrinsic value $= 0$)
- Out-of-the-money option/ OTM (intrinsic value < 0)

Time value: the premium a rational investor would pay over its *current* exercise value (intrinsic value), based on the probability it will increase in value before expiry

Time value reflects the potential gains that may be brought about by the price fluctuations of the underlying asset before the option expires

- The longer the remaining term of an option is, the higher the time value is usually.
- When approaching the expiration date, the value of time will decay at an accelerated rate

Approximation Theorem

Basic concepts

- **Operators**

The definition of the operator is the function of a function. This might sound a little opaque. We can think like this: we often deal with the function which usually maps from $\mathbb{R}$ to $\mathbb{R}$. For the operator, this is actually a mapping from a function to another function. One typical example of the operator is the differential operator, which can take the original function as the input and give the corresponding derivative as the output.

- **Operator semigroup**

An **operator semigroup** $\{P_t\}_{t \geq 0}$ is a family of operators that describes how a system evolves over time. Think of P_t as a "time machine" that tells us the state of our system at time t .

Initial conditions: $P_0 = I$

Semigroup property: $P_{t+s} = P_t P_s$

- **Generator**

The generator of a semigroup is defined as follows:

$$\mathcal{G}f(x) = \lim_{t \rightarrow 0} \frac{P_t f - f}{t}$$

The generator plays a fundamental role in determining the evolution of the semigroup, as guaranteed by the Hille-Yosida theorem.

Here, we consider about the transition rate of P_t and we can write it like this:

$$\frac{d}{dt} P_t f = \lim_{h \rightarrow 0} \frac{P_{t+h} f - P_t f}{h} = \lim_{h \rightarrow 0} \frac{P_t P_h f - P_t f}{h} = P_t \lim_{h \rightarrow 0} \frac{P_h f - f}{h} = P_t (\mathcal{G}f)$$

This gives us the **operator ODE** as follows:

$$\frac{d}{dt} P_t = P_t \mathcal{G}$$

With the initial conditions, we can get the solution to the **operator ODE**: $P_t = e^{t \mathcal{G}}$

In this part, we meet three relatively unfamiliar knowledge points. What we need to know is that according to [Hille-Yosida theorem](#), under specific technical conditions, the generator can uniquely determine the corresponding operator semigroup and further controls the evolution of the stochastic process.

Moreover, the stochastic process we care about are **almost feller process**, which can **automatically satisfy the technical conditions of Hille-Yosida theorem**.

Remark: the conditions of the statement above are not important. For example, you don't need to know what a feller process is because almost all the process applied in the analysis are feller process.

With these basic concepts, we can infer that if we want to realize the approximation of the stochastic process via the CTMC, we need to focus on the two generators of these two process because CTMC is also a feller process and governed by its generator. Therefore, if we can design a CTMC whose Q matrix approximates the generator of the stochastic process closely, we can switch the research focus from the original stochastic process to the constructed CTMC. This is very useful because CTMC is discrete and finite in the state space, which can greatly reduce the research difficulty.

Generator Calculation

Diffusion process

For a typical diffusion process without the jump term which has the following form

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

We can calculate its generator in the following procedure

Step 1: we can apply Ito formula into $f(X_t)$ and we can get the following result

$$\begin{aligned} df(X_t) &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial x}dX_t + \frac{1}{2} \frac{\partial^2 f}{\partial x^2}d[X_t, X_t] \\ &= \mu(X_t) \frac{\partial f}{\partial x}dt + \sigma(X_t) \frac{\partial f}{\partial x}dW_t + \frac{1}{2} \sigma^2(X_t) \frac{\partial^2 f}{\partial x^2}dt \end{aligned}$$

Step 2: we take integral on both sides and the result is as follows:

$$f(X_t) - f(X_0) = \int_0^t \mu(X_s) \frac{\partial f}{\partial x}(X_s)ds + \int_0^t \sigma(X_s) \frac{\partial f}{\partial x}(X_s)dW_s + \int_0^t \frac{1}{2} \sigma^2(X_s) \frac{\partial^2 f}{\partial x^2}(X_s)ds$$

Step 3: we take expectation on both sides given the condition $X_0 = x$ and we can get the following results due to the fact that the expectation of a stochastic integral must be 0, regardless of the property of the integrand

$$\begin{aligned} \mathbb{E}[f(X_t)|X_0 = x] - f(x) &= \mathbb{E} \left[\int_0^t \mu(X_s) \frac{\partial f}{\partial x}(X_s)ds | X_0 = x \right] + \mathbb{E} \left[\int_0^t \sigma(X_s) \frac{\partial f}{\partial x}(X_s)dW_s | X_0 = x \right] \\ &\quad + \mathbb{E} \left[\int_0^t \frac{1}{2} \sigma^2(X_s) \frac{\partial^2 f}{\partial x^2}(X_s)ds | X_0 = x \right] \\ &= \mathbb{E} \left[\int_0^t \mu(X_s) \frac{\partial f}{\partial x}(X_s)ds | X_0 = x \right] + \mathbb{E} \left[\int_0^t \frac{1}{2} \sigma^2(X_s) \frac{\partial^2 f}{\partial x^2}(X_s)ds | X_0 = x \right] \\ &= \mathbb{E} \left[\int_0^t \left(\mu(X_s) \frac{\partial f}{\partial x}(X_s) + \frac{1}{2} \sigma^2(X_s) \frac{\partial^2 f}{\partial x^2}(X_s) \right) ds | X_0 = x \right] \end{aligned}$$

Step 4: According to the definition of the operator of the diffusion process, we need to get the result of $P_t f$. This can be interpreted as the performance of the function f after the transition which lasted for time t , which is equivalent to expectation of the function f at time t conditional on the initial distribution. Therefore, we can get the formula of the infinitesimal generator with a

basic theorem: when $t \rightarrow 0$, the stochastic process X_t will be close to the initial value X_0 .

$$\mathbb{E}[f(X_t) | X_0 = x] - f(x) \approx t \cdot \left(\mu(x) \frac{\partial f}{\partial x}(x) + \frac{1}{2} \sigma^2(x) \frac{\partial^2 f}{\partial x^2}(x) \right)$$

$$\mathcal{G}f(x) = \lim_{t \rightarrow 0} \frac{t \cdot \left(\mu(x) \frac{\partial f}{\partial x} + \frac{1}{2} \sigma^2(x) \frac{\partial^2 f}{\partial x^2} \right)}{t} = \mu(x) \frac{\partial f}{\partial x} + \frac{1}{2} \sigma^2(x) \frac{\partial^2 f}{\partial x^2}$$

Jump Diffusion process

For a stochastic process with the jump term defined as follows:

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t + \int_{\mathbb{R}} h(X_t, y)N(dt, dy)$$

where $N(dt, dx)$ is the Poisson measure, $\nu(dy)$ is the Lévy measure and $h(x, y)$ is the jump amplitude function.

The generator $\mathcal{L}$ of this process is defined as follows:

$$\mathcal{L}f(x) = \mu(x) \frac{\partial f}{\partial x} + \frac{1}{2} \sigma^2(x) \frac{\partial^2 f}{\partial x^2} + \int_{\mathbb{R}} [f(x + h(x, y)) - f(x) - h(x, y)f'(x)] \nu(dy)$$

Before looking at the following derivation, please refer to Appendix C for the knowledge of the jump component.

To derive this generator, we start with following the same procedure as that in the pure diffusion process part

Step 1: For any differentiable test function f , we apply Ito formula and get the following result:

$$df(X_t) = \frac{\partial f}{\partial x}(X_{t-})dX_t + \frac{1}{2} \frac{\partial^2 f}{\partial x^2}(X_{t-})d[X_t, X_t] + \int_{\mathbb{R}} [f(X_{s-} + h(X_t, y)) - f(X_{s-})]N(ds, dy)$$

In fact, this formula can be understood as follows: The first two terms are the same as those in the Ito formula of the diffusion process and the third term is the effect of the jump term. This integral is actually the product of the intensity of each jump term and the number of occurrences at a specific time interval.

Step 2: Substitute dX_t into the formula in Step 1 and unite similar terms, we can get the following result:

$$\begin{aligned} df(X_t) &= \mu(X_t) \frac{\partial f}{\partial x}(X_{t-})dt + \sigma(X_t) \frac{\partial f}{\partial x}(X_{t-})dW_t \\ &\quad + \frac{1}{2} \sigma^2(X_t) \frac{\partial^2 f}{\partial x^2}(X_{t-})dt + \frac{\partial f}{\partial x}(X_{t-}) \int_{\mathbb{R}} h(X_t, y)N(dt, dx) \\ &\quad + \int_{\mathbb{R}} [f(X_{s-} + h(X_t, y)) - f(X_{s-})]N(ds, dy) \\ &= \mu(X_t) \frac{\partial f}{\partial x}(X_{t-})dt + \sigma(X_t) \frac{\partial f}{\partial x}(X_{t-})dW_t + \frac{1}{2} \sigma^2(X_t) \frac{\partial^2 f}{\partial x^2}(X_{t-})dt \\ &\quad + \int_{\mathbb{R}} \left[f(X_{s-} + h(X_t, y)) - f(X_{s-}) + \frac{\partial f}{\partial x}(X_{t-})h(X_t, y) \right] N(ds, dy) \end{aligned}$$

Step 3: Take expectation on both sides, we can get

$$\mathbb{E}[df(X_t)] = \left(\mathbb{E}[\mu(X_t) \frac{\partial f}{\partial x}] + \frac{1}{2} \mathbb{E}[\sigma^2(X_t) \frac{\partial^2 f}{\partial x^2}] \right) dt + \mathbb{E} \left[\int_{\mathbb{R}} \left[f(X_{s-} + h(X_t, y)) - f(X_{s-}) + \frac{\partial f}{\partial x} h(X_t, y) \right] N(ds, dy) \right]$$

Step 4: Finally, we can get the result of the generator by dividing dt at both sides of equation (22):

$$\frac{\mathbb{E}[df(X_t)]}{dt} = \mu(X_t) \frac{\partial f}{\partial x} + \frac{1}{2} \sigma^2(X_t) \frac{\partial^2 f}{\partial x^2} + \int_{\mathbb{R}} \left[f(x + h(x, y)) - f(x) - h(x, y) \frac{\partial f}{\partial x} \right] \nu(dy)$$
$$\mathcal{L}f(x) = \mu(x) \frac{\partial f}{\partial x} + \frac{1}{2} \sigma^2(x) \frac{\partial^2 f}{\partial x^2} + \int_{\mathbb{R}} [f(x + h(x, y)) - f(x) - h(x, y) f'(x)] \nu(dy)$$

Matrix Q of the constructed CTMC

In this subsection, only the structure of matrix Q of the constructed CTMC will be provided. The details of the derivation of this matrix doesn't matter unless you have a keen interest in this derivation. What we need to do is that when facing a completely new stochastic process, we can find the drift term and the diffusion term and make the correct adjustments in the codes.

Setup for the state space X

Before the specific elements in the constructed continuous time Markov chain, we need to realize the transformation from the original continuous data into the discrete state space.

This is actually very easy and convenient to realize in the python codes.

Step 1: Choose the boundary value of the state space (key of the parameter adjustment) — one common choice is the minimum and maximum of the original data.

```
x_min = np.min(data) - 1
x_max = np.max(data) * 1.5
```

Here, we slightly expand the range of the data: subtracting 1 from the minimum and multiplying the maximum by 1.5. This trick makes the binning process more robust to edge values.

Step 2: Divide the entire range into evenly spaced bins according to the desired number of states (uniform grid design method).

```
bins = np.linspace(x_min, x_max, num_bins + 1)
```

This generates $\text{num_bins} + 1$ evenly spaced points from $x_{\min}$ to $x_{\max}$, forming num_bins intervals (i.e., states). These bins define the boundaries of each discrete state.

Step 3: Assign each data point to its corresponding bin index.

```
indices = np.digitize(data, bins) - 1
```

This step maps each continuous data point to an integer label representing its state. Since `np.digitize` returns values starting from 1, we subtract 1 to make the indices start from 0 (i.e., 0-based indexing), which is consistent with the common notation and the python coding.

Step 4: Handle possible edge cases by clipping the indices to ensure they lie within $[0, \text{num_bins}-1]$.

```
indices = np.clip(indices, 0, num_bins - 1)
```

This avoids out-of-range errors when values fall outside the extended bin range (e.g., due to noise or extreme outliers).

Example

To make the discretization process more intuitive, let's consider a simple but vivid example.

Suppose you're monitoring the **temperature in a greenhouse** throughout a day, and you've collected the following continuous measurements (in Celsius):

```
data = np.array([15.3, 17.8, 20.1, 22.7, 24.5, 26.2])
```

Now, suppose you want to **model the greenhouse temperature as a Markov process** with only **3 discrete states**: "Cool", "Mild", and "Hot". You can apply the discretization function to map these continuous values to the discrete state space:

```
indices, bins = discretize_data(data, num_bins=3)
```

Let's walk through what happens step by step.

Step 1: Find the minimum and maximum of the data and extend them slightly:

- Minimum: $15.3 \rightarrow x_{\min} = 15.3 - 1 = 14.3$
- Maximum: $26.2 \rightarrow x_{\max} = 26.2 + 1.5 = 39.3$

Step 2: Generate 4 bin edges for 3 states:

```
bins = np.linspace(14.3, 39.3, 4)
```

This gives:

```
bins = [14.3, 22.97, 31.63, 40.3]
```

So the temperature ranges are:

- State 0 (Cool): [14.3, 22.97)
- State 1 (Mild): [22.97, 31.63)
- State 2 (Hot): [31.63, 40.3]

Step 3: Assign each data point to a bin:

- 15.3 $\rightarrow$ State 0 (Cool)
- 17.8 $\rightarrow$ State 0 (Cool)
- 20.1 $\rightarrow$ State 0 (Cool)
- 22.7 $\rightarrow$ State 0 (Cool)
- 24.5 $\rightarrow$ State 1 (Mild)
- 26.2 $\rightarrow$ State 1 (Mild)

So the resulting discrete state sequence is:

```
indices = [0, 0, 0, 0, 1, 1]
```

Step 4: Output

- `indices`: tells you the temperature state at each time point
- `bins`: provides the exact temperature thresholds used to define each state

This is particularly useful when building **Markov chains**, which require a **finite and discrete state space** for defining transitions.

Diffusion process

For a typical diffusion process without the jump term which has the following form

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The key insight is that we want our discrete CTMC to "behave like" the continuous diffusion process in the limit. This requires matching two fundamental characteristics:

- **Matching the drift:** The expected movement of the CTMC should converge to the drift term
- **Matching the diffusion:** The variance of movement should converge to the diffusion term

Plain Language: Think of this as teaching a "jumping frog" (CTMC) to mimic a "smoothly swimming fish" (diffusion process). The frog can only make discrete hops between lily pads, but we want its overall movement pattern to look just like the fish's smooth swimming. We need to carefully choose how fast and in which direction the frog jumps from each lily pad.

The challenge is: How do we construct the transition rates to achieve this perfect mimicking?

Construction Process

To answer this challenge, we start with a general framework. For a discretized state space, we can express the Q-matrix using arbitrary functions w_n and v_n :

$$Q = \begin{cases} q_{i,i+1} := \frac{1}{x_{i+1}^h - x_i^h} w_n(x_i^h) \\ q_{i,i-1} := \frac{1}{x_i^h - x_{i-1}^h} v_n(x_i^h) \\ q_{i,i} := - \left(\frac{1}{x_{i+1}^h - x_i^h} w_n(x_i^h) + \frac{1}{x_i^h - x_{i-1}^h} v_n(x_i^h) \right) \\ q_{i,j} = 0 \quad \forall j \neq i, i-1, i+1 \end{cases}$$

where w_n and v_n are arbitrary functions that we need to determine.

Plain Language: Here, $w_n(x_i^h)$ controls the "rightward jumping intensity" from state i , and $v_n(x_i^h)$ controls the "leftward jumping intensity" from state i . The question is: how should we choose these functions so that our CTMC behaves like the original diffusion process?

It is evident that if we choose w_n and v_n correctly, the CTMC will converge to the diffusion process as the grid becomes finer. As a consequence, how can we choose the correct two functions?

As mentioned above, the two process are governed by their own generator, so if we allow the generator of the constructed CTMC to approximate that of the original stochastic process closely enough, the approximation can be realized.

Knowing about this point, we can apply the infinitesimal generator on the test function f and get the following result:

$$\begin{aligned} Qf(x)|_{x=x_i^h} &= (f(x_{i-1}^h)q_{i,i-1} + f(x_{i+1}^h)q_{i,i+1} + f(x_i^h)q_{i,i}) \\ &= ([f(x_{i-1}^h) - f(x_i^h)]q_{i,i-1} + [f(x_{i+1}^h) - f(x_i^h)]q_{i,i+1}) \\ &= \left([f(x_{i-1}^h) - f(x_i^h)] \frac{1}{x_i^h - x_{i-1}^h} v_n(x_i^h) + [f(x_{i+1}^h) - f(x_i^h)] \frac{1}{x_{i+1}^h - x_i^h} w_n(x_i^h) \right) \end{aligned}$$

Remark: Q here is a matrix whose size is $N \times N$ (N is the number of states) and the input of the test function is $N \times 1$. In the analysis, we pick one specific element of the the product($N \times 1$) and get the convergence conditions.

The next step is that we apply Taylor expansion of the two parts: $f(x_{i-1}^h) - f(x_i^h)$ and $f(x_{i+1}^h) - f(x_i^h)$. This is an extremely **critical step** because this is a key source of the error because we do the truncation, ignoring the high order terms. The result is as follows:

$$\begin{aligned} Qf(x)|_{x=x_i^h} &= \left(- \left[f'(x_i^h) + \frac{1}{2} f''(x_i^h)(x_{i-1}^h - x_i^h) + o(x_{i-1}^h - x_i^h) \right] v_n(x_i^h) \right. \\ &\quad \left. + \left[f'(x_i^h) + \frac{1}{2} f''(x_i^h)(x_{i+1}^h - x_i^h) + o(x_{i+1}^h - x_i^h) \right] w_n(x_i^h) \right) \\ &\approx (f'(x_i^h)(w_n(x_i^h) - v_n(x_i^h)) \\ &\quad + \frac{1}{2} f''(x_i^h) [(x_{i+1}^h - x_i^h)w_n(x_i^h) - (x_{i-1}^h - x_i^h)v_n(x_i^h) + o(x_{i+1}^h - x_{i-1}^h)]) \end{aligned}$$

To ensure our CTMC converges to the diffusion process, we need two critical conditions:

1. **Drift Matching Condition:** $\lim_{n \rightarrow \infty} (w_n(x_i^h) - v_n(x_i^h)) = \mu(x_i^h)$
2. **Variance Matching Condition:** $\lim_{n \rightarrow \infty} ((x_{i+1}^h - x_i^h)w_n(x_i^h) - (x_i^h - x_{i-1}^h)v_n(x_i^h)) = \sigma^2(x_i^h)$

Plain Language: These two conditions are the "magic requirements" - they ensure our jumping process (CTMC) has the same long-term behavior as the smooth diffusion process. The first condition says "the average jumping tendency should match the drift", and the second says "the randomness level should match the volatility".

After solving the convergence conditions (refer to Appendix B for the detailed derivation), the transition rate matrix is as follows:

$$Q := \begin{cases} q_{i,i+1} = \frac{h_{i-1}\mu(x_i^h) + \sigma^2(x_i^h)}{h_i(h_i + h_{i-1})} \\ q_{i,i-1} = \frac{-h_i\mu(x_i^h) + \sigma^2(x_i^h)}{h_{i-1}(h_i + h_{i-1})} \\ q_{i,i} = -(q_{i,i+1} + q_{i,i-1}) \end{cases}$$

Here, some notation needs to be mentioned

- x_i^h : the i th state in the state space
- h_i : the distance between the i th and $i+1$ th state (same in the uniform grid design method)

Moreover, there is also another common version of matrix Q , whose structure is as follows:

$$Q = \begin{cases} q_{i,i+1} := \left(\frac{1}{h_i} \mu_+(x_i^h) + \frac{\sigma^2(x_i^h) - (h_{i-1}\mu_-(x_i^h) + h_i\mu_+(x_i^h))}{h_i(h_i + h_{i-1})} \right) \\ q_{i,i-1} := \left(\frac{1}{h_{i-1}} \mu_-(x_i^h) + \frac{\sigma^2(x_i^h) - (h_{i-1}\mu_-(x_i^h) + h_i\mu_+(x_i^h))}{h_{i-1}(h_i + h_{i-1})} \right) \\ q_{i,i} := -(q_{i,i-1} + q_{i,i+1}) \\ q_{i,j} = 0 \quad \forall j \neq i, i-1, i+1 \end{cases}$$

This is actually the same as the original version, only decomposing the drift term into $\mu_+(x_i^h) - \mu_-(x_i^h)$, where $\mu_+(x_i^h)$ is $\max(0, \mu(x_i^h))$ and $\mu_-(x_i^h)$ is $\max(0, -\mu(x_i^h))$. If you substitute this expression into the original version, you can get the updated version of this matrix.

Example of finding drift and diffusion

Actually, for the application in the real scenario, we don't need to completely understand the theoretical part above. We just need to find the drift and the diffusion correctly and transform the mathematical formula into the python codes. Some examples are provided here to help readers understand how to find the two critical terms.

Take the 1D - CEV process as one example, the process is as follows:

$$\frac{dX_t}{X_t} = \sigma \left(\frac{X_t}{X_0} \right)^\beta dW_t$$

In this model, the drift term and the diffusion term are as follows:

Drift: 0

Diffusion: $\sigma \left(\frac{X_t}{X_0} \right)^\beta$

Take the Ornstein–Uhlenbeck process as another example, the process is as follows:

$$dX_t = \theta(\mu - X_t)dt + \sigma\sqrt{X_t}dW_t$$

In this model, the drift term and the diffusion term are as follows:

Drift: $\theta(\mu - X_t)$

Diffusion: $\sigma\sqrt{X_t}$

Remark: The diffusion term here is equal to the term $\sigma\sqrt{X_t}$ and consistent with the term **diff** written in the python codes.

Jump Diffusion Process

The diffusion process with a jump term actually can be decomposed into two processes: the normal diffusion process and a pure jump process, so the infinitesimal generator of this process can also be decomposed into two parts: the normal part (derived in the last subsection) and the jump part

$$Q = Q^D + Q^J$$

where:

- Q^D = **continuous operator** (handle the smooth diffusion part)
- Q^J = **jump operator** (handle the sudden jumps part)

However, there are restrictions about the construction of the two parts of the Markov chain. We need to follow the moment matching rules, which can be expressed in the following mathematical form. Here, we assume that the original process is denoted as X_t , which follows the following typical stochastic process:

$$dX_t = \mu(t, X_t)dt + \sigma(t, X_t)dW_t + dJ_t$$

where:

J_t is a compound Poisson process, whose distribution of the jump size is F_Y .

Therefore, the moment matching rule can be written as follows:

$$\sum_{j=1}^N \Lambda_{ij}^D(x_j - x_i) + \sum_{j=1}^N \Lambda_{ij}^J(x_j - x_i) = E\left[\frac{dX_t}{dt} | F_t\right] = \mu(t_i, X_i) + \frac{E(dJ_t) | X_t = x_i}{dt}$$

$$\sum_{j=1}^N \Lambda_{ij}^D(x_j - x_i)^2 + \sum_{j=1}^N \Lambda_{ij}^J(x_j - x_i)^2 = E\left[\frac{(dX_t)^2}{dt} | F_t\right] = \sigma^2(t_i, X_i) + \frac{E(dJ_t)^2 | X_t = x_i}{dt}$$

Construction Process

In this process, we only need to focus on the construction of the matrix Q^J because Q^C is derived in the former section.

To ensure that the Markov chain approximation of a jump-diffusion process is valid, we must guarantee that:

- The **first** and **second instantaneous moments** of the jump process in the discrete approximation match those of the original continuous-time process.
 1. The expectation of the jump: $\mu_J(x_i) := \int_{\mathbb{R}} h(x_i, y) \nu(x_i, dy)$
 2. The variance of the jump: $\sigma_J^2(x_i) := \int_{\mathbb{R}} h^2(x_i, y) \nu(x_i, dy)$
- Another important **convergence condition** is finite variance: $\int_{\mathbb{R}} h^2(x_i, y) \nu(dy) < \infty$

The derivation process is a little hard, so if not interested, directly use the following form of the transition rate matrix. If interested, please refer to Appendix D for the details.

$$Q := Q^C + Q^J = \begin{cases} q_{i,i+1} := \left(\frac{1}{h_i} \mu_+(x_i^h) + \frac{\sigma^2(x_i^h) - (h_{i-1} \mu_-(x_i^h) + h_i \mu_+(x_i^h))}{h_i(h_i + h_{i-1})} \right) \\ q_{i,i-1} := \left(\frac{1}{h_{i-1}} \mu_-(x_i^h) + \frac{\sigma^2(x_i^h) - (h_{i-1} \mu_-(x_i^h) + h_i \mu_+(x_i^h))}{h_{i-1}(h_i + h_{i-1})} \right) \\ q_{i,j} = \int_{h^{-1}(x_i^h, x_j^h - x_i^h)} \nu(x_i^h, dy), \quad \forall j \neq i, i-1, i+1 \\ q_{i,i} := - \left(q_{i,i-1} + q_{i,i+1} + \sum_j q_{i,j} \right) \end{cases}$$

Regime - Switching Jump Diffusion Process

To model the joint dynamics of asset price and market regime, we approximate the continuous-time stochastic process with a finite-state continuous-time Markov chain (CTMC). This approach captures both the price movement within each regime and the transitions between regimes.

Core Idea:

We construct a block-structured generator matrix Q that jointly governs the evolution of price and regime. Each diagonal block $Q^{(m)}$ encodes the dynamics under regime m , while off-diagonal blocks $Q^{(m_1 m_2)}$ represent regime-switching transitions. The entire matrix is designed to be row-sum-zero, ensuring the satisfaction of the basic properties of a CTMC.

Construction Process

The generator matrix $Q \in \mathbb{R}^{K \times K}$ (with $K = M \cdot N$) is assembled in three steps:

Step 1: Intra-Regime Diffusion Blocks

For each regime $m = 1, \dots, M$:

- Construct an $N \times N$ tridiagonal matrix $\Lambda^{(m)}$ that discretizes the CEV diffusion generator via **moment matching** on a log-price grid $\{x_n\}$.
 - Interior nodes ($n = 2, \dots, N-1$):**

$$\begin{aligned}\Lambda_{n,n-1}^{(m)} &= \frac{D_n}{h_{n-1}(h_n + h_{n-1})} \\ \Lambda_{n,n+1}^{(m)} &= \frac{D_n}{h_n(h_n + h_{n-1})} \\ \Lambda_{n,n}^{(m)} &= -\left(\Lambda_{n,n-1}^{(m)} + \Lambda_{n,n+1}^{(m)}\right)\end{aligned}$$

where $h_n = x_{n+1} - x_n$ and $D_n = \text{Diffusion Part}$.

- Boundary nodes ($n = 1, N$):**

Apply reflecting boundaries:

$$\Lambda_{1,2}^{(m)} = \frac{D_1}{h_1^2}, \quad \Lambda_{1,1}^{(m)} = -\Lambda_{1,2}^{(m)} \quad \Lambda_{N,N-1}^{(m)} = \frac{D_N}{h_{N-1}^2} \quad \Lambda_{N,N}^{(m)} = -\Lambda_{N,N-1}^{(m)}$$

Actually, this step for each specific regime is identical to the procedure in the subsection above and we just need to repeat this procedure for M (number of states) times.

Step 2: Inter-Regime Switching Blocks

For all $m_1 \neq m_2$:

- Set the off-diagonal block as

$$Q^{m_1 m_2} = \lambda_{m_1 m_2} I_N$$

meaning a transition from any price node x_n in regime m_1 to the **same** price node x_n in regime m_2 occurs at rate $\lambda_{m_1 m_2}$.

Step 3: Diagonal Block Correction

For each regime m :

- Adjust the diagonal block to account for total outflow to other regimes:

$$Q^{mm} = \Lambda^{(m)} - \left(\sum_{j \neq m} \lambda_{mj} \right) I_N.$$

- This ensures every row of Q sums to zero, satisfying the defining property of a CTMC generator.

Parameter Estimation

In the estimation of the parameters, we apply the Maximum Likelihood Estimation (MLE) method. Therefore, the question automatically lies in the construction of the likelihood function.

As we all know, the basic form of the likelihood function is as follows:

$$lik(\theta) = P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n)$$

Step 1: Transfer the research focus from the original data to the corresponding state labels.

In a mathematical form, we can express the likelihood function as follows:

$$lik(\theta) = P(X_1^h = x_1^h, X_2^h = x_2^h, \dots, X_n^h = x_n^h)$$

Actually, this is very intuitive because the constructed CTMC can be seen as a substitute for the original data and is easy to deal with compared to the original data.

Step 2: Simplify the expression for the likelihood function via Markov property mentioned in the section Preliminary Knowledge

$$\begin{aligned} lik(\theta) &= P(X_1^h = x_1^h, X_2^h = x_2^h, \dots, X_n^h = x_n^h) \\ &= P(X_n^h = x_n^h | X_1^h = x_1^h, X_2^h = x_2^h, \dots, X_{n-1}^h = x_{n-1}^h) * P(X_1^h = x_1^h, X_2^h = x_2^h, \dots, X_{n-1}^h = x_{n-1}^h) \\ &= P(X_n^h = x_n^h | X_{n-1}^h = x_{n-1}^h) * P(X_1^h = x_1^h, X_2^h = x_2^h, \dots, X_{n-1}^h = x_{n-1}^h) \\ &= P(X_n^h = x_n^h | X_{n-1}^h = x_{n-1}^h) \dots P(X_2^h = x_2^h | X_1^h = x_1^h) P(X_1^h = x_1^h) \end{aligned}$$

Step 3: The conditional probabilities in the likelihood function correspond to transition probabilities in the matrix $P_{\{\Delta t\}} = e^{\{\Delta t Q\}}$, where Δt is the time interval between consecutive observations. For discrete observations at times $t_1, t_2, \dots, t_n$ we have:

$$P(X_{i+1}^h = s_j | X_i^h = s_k) = [e^{\Delta t Q(\theta)}]_{k,j}$$

where s_k, s_j are the discrete states corresponding to X_i^h and X_{i+1}^h respectively.

Remark: The parameters to be estimated are the key components of the transition rate matrix. For instance, in the basic CEV model, the parameters σ and β play a key role in the construction of the transition rate matrix.

Step 4: We need to construct the complete likelihood function

For a sequence of discretized observations $\{X_1^h, X_2^h, \dots, X_n^h\}$ with corresponding state indices $\{s_1, s_2, \dots, s_n\}$, the likelihood function becomes:

$$L(\theta) = \pi(s_1) \prod_{i=1}^{n-1} [P_{\Delta t}(\theta)]_{s_i, s_{i+1}}$$

where:

- $\pi(s_1)$ is the initial distribution
- $P_{\{\Delta t\}}(\theta) = e^{\{\Delta t Q(\theta)\}}$ is the transition probability matrix
- θ represents the parameter vector to be estimated

Step 5: Log-Likelihood Function

Taking logarithms for numerical stability:

$$\ell(\theta) = \log \pi(s_1) + \sum_{i=1}^{n-1} \log [P_{\Delta t}(\theta)]_{s_i, s_{i+1}}$$

Step 6: Optimization

The MLE estimator is obtained by:

$$\hat{\theta} = \arg \max_{\theta} \ell(\theta)$$

This requires:

1. Numerical computation of matrix exponential $e^{\Delta t Q(\theta)}$
2. Gradient-based optimization algorithms
3. Appropriate initial parameter values

Pricing Process

The key challenge posed by American options is that they can be exercised at ANY time before maturity. This creates an **optimal exercise problem**: at each moment, should you exercise now or wait?

The Fundamental Principle

At any time t , the American option value is:

$$V_t = \max\{\text{Immediate Exercise Value}, \text{Continuation Value}\}$$

Plain Language: At each moment, you compare:

- What you'd get if you exercise right now
- What you expect to get if you wait

Intuitively, you will choose whichever is larger!

Example - The Gym Membership Analogy: Imagine you have a coupon that lets you join a gym at a fixed price of \$50. The gym's membership price fluctuates:

- Today it costs 60 (you'd save 10 by using the coupon now)
- But next month it might cost 80 (you'd save 30 if you wait)
- Or it might drop to 40 (your coupon becomes worthless)

You must decide: use the coupon now for a guaranteed \$10 savings, or wait and hope for bigger savings?

After understanding the essence of the American options pricing, we will enter the framework of CTMC.

The CTMC approach discretizes both price and time dimensions:

- **Price dimension:** Continuous prices → Finite states (e.g., 500 price levels)
- **Time dimension:** Continuous time → Discrete steps (e.g., 1000 time points)

This transforms an intractable continuous problem into a manageable discrete one that can be solved via backward dynamic programming algorithm.

Before entering the framework of the backward dynamic programming algorithm, readers should read the following insight carefully to prevent the occurrence of a very critical error.

Critical Insight: Two Different Time Steps

A crucial but often confusing aspect of the implementation involves two distinct time discretizations:

1. Parameter Estimation Time Step ($\Delta t = 1/252$)

When calibrating the CTMC model from historical futures data:

```
opt_params = fit_params_futures(state_seq, bins, 1/252, F_0, x0)
```

Here, `dt = 1/252` represents one trading day. This choice is driven by data availability—futures prices are typically observed daily. The maximum likelihood estimation uses these daily transitions to estimate the model parameters (σ , β).

2. Option Pricing Time Step ($dt = t_0/T/N_t$)

When actually pricing the option:

```
CTMC_parameters = {
    'N_t': 1000,
    'dt': t_0/T/N_t,
}
```

This time step is much finer and is calculated based on:

- `t_0`: Time to maturity of the specific option
- `N_t`: Number of time steps chosen for pricing accuracy (e.g., 1000).

Key Takeaway: Make sure not to confuse these two dt 's — the first is for *calibration*, the second is for *pricing*.

Backward Dynamic Programming Algorithm

Step 1: Initialize the Grid

```
V = np.zeros((N_s, N_t+1)) # value matrix: states x time
```

This creates a 2D grid where:

- Rows represent different price states
- Columns represent different time points
- Each cell $V[i,j]$ will store the option value at price state i and time j

Step 2: Set Boundary Conditions (Terminal Payoff)

At maturity, the futures price will have N possibilities and the holder can only choose to exercise this option. As a consequence, $V(T, X_T)=[f(x_1), f(x_2), \dots, f(x_N)]$, which is also the starting point of the whole inverse for loop. f represents the value function. For example, the function f of a call option is $\max(F-K, 0)$.

In python codes, we often realize this step as follows:

```
if option_type == 'put':
    exercise_value = np.maximum(K - bins_centers, 0)
else:
    exercise_value = np.maximum(bins_centers - K, 0)
V[:, N_t] = exercise_value
```

Note: `exercise_value` is only a function of the current price state, so it does not change with time.

For example, with a put option ($K=100$) and price states $[90, 100, 110]$:

- $V[0, N_t] = \max(100-90, 0) = 10$

- $V[1, N_t] = \max(100-100, 0) = 0$
- $V[2, N_t] = \max(100-110, 0) = 0$

Step 3: Backward Induction

At each previous time point, we can treat the current option value as the maximum of the value of immediately exercising this option and the value which you can get from waiting for a time interval. The mathematical form of this value is as follows:

$$\mathbf{V}(t_i, \widetilde{\mathbf{X}}_{t_i}) = \begin{bmatrix} \max\{f(\mathbf{x}_1), \mathbb{E}[e^{-r\Delta t} V(t_{i+1}, \widetilde{\mathbf{X}}_{t_{i+1}}) \mid \widetilde{\mathbf{X}}_{t_i} = \mathbf{x}_1]\}, \\ \vdots \\ \max\{f(\mathbf{x}_N), \mathbb{E}[e^{-r\Delta t} V(t_{i+1}, \widetilde{\mathbf{X}}_{t_{i+1}}) \mid \widetilde{\mathbf{X}}_{t_i} = \mathbf{x}_N]\} \end{bmatrix}$$

$$= \max\{f(\mathbf{X}), e^{-r\Delta t} \mathbf{P}(\Delta t) \mathbf{V}(t_{i+1}, \widetilde{\mathbf{X}}_{t_{i+1}})\}$$

where:

- $\mathbf{P}(\Delta t) = \exp(Q\Delta t)$ is the transition probability matrix obtained from Q
- $e^{-r\Delta t}$ discounts the value back to the current step

In python codes, we often realize this step as follows:

```
for i in range(N_t-1, -1, -1):
    continuation_value = np.exp(-r_f * dt) * P @ V[:, i + 1]
    if option_am_eu == 'am':
        V[:, i] = np.maximum(exercise_value, continuation_value)
    else: # European
        V[:, i] = continuation_value
```

Key Insight:

- For **American** options we take the maximum (because you can exercise any time)
- For **European** options we use continuation value only (because early exercise is not allowed)

Understanding the Continuation Value Calculation

The line `continuation_value = np.exp(-r_f * dt) * P @ V[:, i + 1]` performs three operations:

1. `P @ V[:, i + 1]`: Matrix multiplication computes expected future values

- $P[i, j]$ = probability of moving from state i to state j
- $V[j, i+1]$ = option value at state j in the next period
- The product gives expected value from each starting state

2. `np.exp(-r_f * dt)`: Discounts future values to present

- r_f = risk-free rate
- dt = time step (option pricing time step, not daily!)

3. Result: Vector of continuation values for each price state

A Concrete Example

Consider pricing an American put with $K=100$ at time step 50 (midway to expiry):

Current price states: [95, 100, 105] Exercise values: [5, 0, 0]

Future values (from previous backward step): $V[:, 51] = [7, 2, 0.5]$

Transition matrix P (simplified):

	95	100	105
95	[0.6	0.3	0.1]
100	[0.3	0.4	0.3]
105	[0.1	0.3	0.6]

Calculate continuation values:

- From 95: $0.995 \times (0.6 \times 7 + 0.3 \times 2 + 0.1 \times 0.5) = 0.995 \times 4.85 = 4.83$
- From 100: $0.995 \times (0.3 \times 7 + 0.4 \times 2 + 0.3 \times 0.5) = 0.995 \times 3.05 = 3.03$
- From 105: $0.995 \times (0.1 \times 7 + 0.3 \times 2 + 0.6 \times 0.5) = 0.995 \times 1.60 = 1.59$

American option values at time 50:

- State 95: $\max(5, 4.83) = 5 \rightarrow$ Exercise optimal
- State 100: $\max(0, 3.03) = 3.03 \rightarrow$ Hold optimal
- State 105: $\max(0, 1.59) = 1.59 \rightarrow$ Hold optimal

Step 4: Extract Today's Price

Finally, we compute today's option price by finding which state the current futures price belongs to and taking the expectation of next-step value:

```
F_t = model_parameters['F_t'] # Current futures price
Ft_index = np.digitize(F_t, bins_centers) # Find corresponding state
V_t = P[Ft_index,:] @ V[:,1] # Expected value from current state
```

This step maps today's futures price to the closest state in the grid, then takes the probability-weighted expected value of the option after one time step.

Examples

In this section, we will show how to apply the procedures above into the parameter estimation of the CEV and CEV-DEJD model.

Example 1

Model Specification

Under the CEV model, the dynamics of the futures prices F_t follows

$$\frac{dF_t}{F_t} = \sigma \left(\frac{F_t}{F_0} \right)^\beta dW_t$$

where $(W_u)_{u \geq t}$ is a standard Brownian motion, σ is the volatility parameter, and β is the elasticity factor.

Approximation Process

In this subsection, we will construct the transition rate matrix for this markov chain. Since CEV is a simple process without the appearance of the jump term, there only exists the matrix Q^C . Then, what we need to do is just to find the drift part and the diffusion part. Specifically, the drift part in CEV process is 0, so we just need to focus on the diffusion part, which can be expressed as

follows:

$$\sigma^2(F_i^h) = \sigma^2(F_i^h)^{2\beta+2} / (F_0)^{2\beta}$$

Because this is a pure diffusion process without the impact of jump term, the non-zero elements are only at the upper diagonal, lower diagonal and the diagonal.

Then, we just substitute this part into the formula given in the former sections and get the following result:

$$\begin{aligned} q_{i,i+1}^D &= \frac{\sigma^2(F_i^h)^{2\beta+2} / (F_0)^{2\beta}}{h_i(h_i + h_{i-1})} \\ q_{i,i-1}^D &= \frac{\sigma^2(F_i^h)^{2\beta+2} / (F_0)^{2\beta}}{h_{i-1}(h_i + h_{i-1})} \\ q_{i,i}^D &= -(q_{i,i+1}^C + q_{i,i-1}^C) \end{aligned}$$

Parameter Estimation

After getting the matrix Q of the constructed CTMC, we can use the relationship between P and Q — $P = e^{tQ}$ — to finish the construction of the likelihood function by finding the corresponding conditional probability.

This part is very straightforward and no extra explanation will be provided here. If still confused about this part, refer to the former subsection "Parameter Estimation".

Pricing Process

After we get the estimated parameters, we can use the backward dynamic programming algorithms to construct the price matrix. Combined with the transition probability matrix, we can get the weighted average price of each time point as the final price of each discrete time point.

After setting the basic information about the selected option(strike price, tau and so on), we can select the underlying model(CEV) and get the pricing results.

Example 2

Model Specification

Under the CEV-DEJD model, the dynamics of the futures prices F_t follows:

$$\frac{dF_t}{F_t} = -\lambda\zeta dt + \sigma \left(\frac{F_t}{F_0} \right)^\beta dW_t + d \left(\sum_{i=1}^{N_t} (e^{Y_i} - 1) \right)$$

Detailed Parameter Explanations:

σ : Base volatility parameter. This represents the annualized volatility when $F_t = F_0$.

β : Elasticity parameter. This controls how volatility changes with the price level:

- $\beta = 0$: Geometric Brownian Motion (constant volatility σ)
- $\beta > 0$: Leverage effect (volatility increases when price increases, typical for stocks)
- $\beta < 0$: Inverse leverage effect (volatility decreases when price increases, typical for commodities)

\$F_0\$: Reference price level. This is the scaling factor for the CEV volatility specification, typically set to initial futures price or long-term average price.

\$\lambda\$: Jump arrival intensity. This is the average number of jumps per unit time. For state-dependent intensity: $\lambda(F_t) = \lambda \left(\frac{F_t}{F_0} \right)^\beta$. (e.g., $\lambda = 5$ means 5 jumps per year on average).

\$N_t\$: Poisson counting process. This counts the number of jumps up to time t . For state-dependent intensity, it becomes a doubly stochastic Poisson process with intensity $\lambda(F_t)$.

\$\{Y_i\}\$: Independent double exponential random variables with the density function:

$$f_Y(y) = p\eta_1 e^{-\eta_1 y} \mathbf{1}_{y \geq 0} + (1-p)\eta_2 e^{\eta_2 y} \mathbf{1}_{y < 0}$$

\$p\$: Probability of positive jump, with $0 \leq p \leq 1$. This represents the proportion of jumps that are upward movements.

\$\eta_1\$: Decay rate for positive jumps, with $\eta_1 > 1$ (required for $\mathbb{E}[e^Y] < \infty$ under risk-neutral measure). The mean positive jump size is $1/\eta_1$.

\$\eta_2\$: Decay rate for negative jumps, with $\eta_2 > 0$. The mean negative jump size (in absolute value) is $1/\eta_2$.

\$\zeta\$: Jump compensator (martingale correction term):

$$\zeta = \mathbb{E}[e^{Y_i}] - 1 = \frac{p\eta_1}{\eta_1 - 1} + \frac{(1-p)\eta_2}{\eta_2 + 1} - 1$$

Detailed Derivation of \$\zeta\$:

We need to calculate $\mathbb{E}[e^Y]$ for the double exponential distribution:

$$\begin{aligned} \mathbb{E}[e^Y] &= \int_{-\infty}^{\infty} e^y f_Y(y) dy \\ &= \int_0^{\infty} e^y \cdot p\eta_1 e^{-\eta_1 y} dy + \int_{-\infty}^0 e^y \cdot (1-p)\eta_2 e^{\eta_2 y} dy \\ &= p\eta_1 \int_0^{\infty} e^{-(\eta_1-1)y} dy + (1-p)\eta_2 \int_{-\infty}^0 e^{(\eta_2+1)y} dy \end{aligned}$$

Now evaluate each integral:

For the first integral ($\eta_1 > 1$ required):

$$\int_0^{\infty} e^{-(\eta_1-1)y} dy = \left[-\frac{1}{\eta_1 - 1} e^{-(\eta_1-1)y} \right]_0^{\infty} = \frac{1}{\eta_1 - 1}$$

For the second integral ($\eta_2 > 0$):

$$\int_{-\infty}^0 e^{(\eta_2+1)y} dy = \left[\frac{1}{\eta_2 + 1} e^{(\eta_2+1)y} \right]_{-\infty}^0 = \frac{1}{\eta_2 + 1}$$

Thus:

$$\mathbb{E}[e^Y] = p\eta_1 \cdot \frac{1}{\eta_1 - 1} + (1-p)\eta_2 \cdot \frac{1}{\eta_2 + 1} = \frac{p\eta_1}{\eta_1 - 1} + \frac{(1-p)\eta_2}{\eta_2 + 1}$$

Therefore:

$$\zeta = \mathbb{E}[e^Y] - 1 = \frac{p\eta_1}{\eta_1 - 1} + \frac{(1-p)\eta_2}{\eta_2 + 1} - 1$$

The term $-\lambda \eta dt$ is crucial because it ensures that under the risk-neutral measure, the futures price process is a martingale. This means $\mathbb{E}^{\mathbb{Q}}[dF_t | \mathcal{F}_t] = 0$, which is required for no-arbitrage pricing.

Complete Parameter Vector: $\theta = (\sigma, \beta, \lambda, p, \eta_1, \eta_2)$ with constraints: $\sigma > 0$, $\beta \in \mathbb{R}$, $\lambda > 0$, $0 \leq p \leq 1$, $\eta_1 > 1$, $\eta_2 > 0$.

Approximation Process

The CTMC approximation for CEV-DEJD follows a **two-step decomposition**:

1. **Discretize the state space** (price levels)
2. **Construct the generator matrix** $Q = Q^D + Q^J$
 - Q^D : Diffusion component (handles continuous price movements)
 - Q^J : Jump component (handles sudden price jumps)

The key challenge is ensuring the **combined CTMC preserves**:

- **Martingale property** under risk-neutral measure
- **Correct instantaneous moments** (drift and variance)
- **Jump size distribution** (double exponential)

Step 1: State Space Discretization

This step transforms the continuous futures price F_t into a finite set of discrete states $\{x_1^h, x_2^h, \dots, x_N^h\}$, which will form the state space of our CTMC.

Procedure (Recap from "Setup for the state space X"):

1. Determine the minimum and maximum of the observed or relevant price range, extending it slightly for robustness.
2. Divide this range into N equally or non-equally spaced intervals (bins). The midpoints or representative points of these bins become our discrete states x_i^h .
3. Map each observed price data point to the index of the bin it falls into, creating a sequence of discrete state labels.

Key Consideration for CEV-DEJD:

- Due to the possibility of large jumps, the state space **must be wide enough** to cover extreme price movements with high probability (e.g., ± 5 standard deviations of the jump distribution).
- For the state-dependent intensity $\lambda(F_t) = \lambda (F_t/F_0)^\beta$, the grid point x_i^h is used to evaluate the local jump intensity.

Once the state space is defined and data is mapped to it, we proceed to construct the generator matrix.

Step 2: Constructing the Generator Matrix Q

With the discrete state space $\{x_i^h\}$ in place, we now build the generator matrix Q that defines the transition rates between these states. Following the decomposition $Q = Q^D + Q^J$, we construct each component separately and then combine them.

Part A: Constructing the Jump Component Q^J

Construction Blueprint for Q^J :

- Conceptual Mapping:** Continuous jumps $\rightarrow$ Discrete state transitions
- Rate Calculation:** For each state i , compute jump intensity to all other states j
- Case Handling:** Separate integration based on jump direction (positive/negative/mixed)
- Matrix Assembly:** Fill Q^J matrix and ensure row sums to zero

The core formula for q_{ij}^J is:

$$q_{ij}^J = \lambda \left(\frac{x_i^h}{F_0} \right)^\beta \cdot \mathbb{P}(\text{jump from state } i \text{ lands in interval of state } j)$$

Notation Summary for Jump Construction:

Symbol	Meaning	Role in Q^J Construction
x_i^h	Discrete price level of state i	Starting state for a jump
$\alpha_{i(j)}$	Relative jump size needed to reach the <i>lower boundary</i> of state j from state i	Defines integration interval $[\alpha_{i(j-1)}, \alpha_{i(j)}]$
$Z = e^Y - 1$	Relative jump size random variable	The variable we integrate over; $Z \in (-1, \infty)$
$f_Z(z)$	Probability density function of Z	Integrand; derived from the double-exponential density of Y
$\lambda (x_i^h/F_0)^\beta$	State-dependent jump intensity at x_i^h	Scalar multiplier for the transition rate

The transition rate q_{ij}^J essentially computes: **"Intensity of jumps from state i "** $\times$ **"Probability that a jump's relative size lands in the interval corresponding to state j "**.

Step 1: Understanding the jump mechanism

1.1 How Jumps Occur

The jump term in the CEV-DEJD model: $d \left(\sum_{i=1}^{N_t} (e^{Y_i} - 1) \right)$

Plain Language Explanation:

- N_t is a "news event counter": on average, λ events occur per unit time
- Each time an event occurs, a random Y_i is drawn (can be positive or negative)
- The price jumps from F_t to $F_t \cdot e^{Y_i}$

Concrete Example:

- Current stock price: $F_t = 100$ dollars
- Draw $Y_i = 0.1$, then $e^{Y_i} = e^{0.1} \approx 1.105$

- New stock price: $F_t \cdot e^{Y_i} = 100 \times 1.105 = 110.5$ dollars
- Jump size: $F_t(e^{Y_i} - 1) = 100 \times 0.105 = 10.5$ dollars
- Relative jump size: $e^{Y_i} - 1 = 10.5\%$

• 1.2 Jump Amplitude Distribution

Y_i follows a double exponential distribution: $f_Y(y) = p\eta_1 e^{-\eta_1 y} \mathbf{1}_{\{y \geq 0\}} + (1-p)\eta_2 e^{\eta_2 y} \mathbf{1}_{\{y < 0\}}$

Plain Language Explanation:

- With probability p , it's "good news" ($y \geq 0$), stock price goes up
- With probability $(1-p)$, it's "bad news" ($y < 0$), stock price goes down
- η_1, η_2 control the **intensity** of the jumps

Vivid Analogy: This is like flipping a coin to determine the news type, then deciding the jump size based on the severity of the news.

Step 2: The Bridge from Continuous to Discrete

• 2.1 The Discretization Challenge

Problem: In the continuous model, prices can jump to any value. But our CTMC only has finite states $x_1^h, x_2^h, \dots, x_N^h$.

Solution Strategy: Map "jump target intervals" to "nearest discrete states."

• 2.2 State Mapping Relationship

Suppose we're currently in state x_i^h , after jumping we should reach: $\text{Target value} = x_i^h \cdot e^Y$

But this target will, in general, **not** land exactly on a discrete grid point.

Mapping Rule: If $x_i^h \cdot e^Y$ is closest to x_j^h , we consider it jumps to state j

• 2.3 Introducing the $\alpha_i(j)$ Function

For convenience in later formulas, define:

$$\alpha_i(j) = \frac{1}{2} \left(\frac{x_j^h + x_{j+1}^h}{x_i^h} \right) - 1$$

Physical meaning: $\alpha_i(j-1)$ is the **relative jump amplitude** from state i to the **lower boundary** of state j .

Example:

- Current state: $x_i^h = 100$
- State j has $x_{j-1}^h = 110$, $x_j^h = 120$
- Midpoint below j : $\frac{1}{2}(x_{j-1}^h + x_j^h) = 115$
- Then $\alpha_i(j) = \frac{115}{100} - 1 = 0.15$ (15% increase)

Key relationship:

We denote $e^{Y_i} - 1$ as a new random variable Z , which currently represents the meaning of the relative jump size, This is consistent with the meaning of the $\alpha_i(j)$ function.

As a consequence, our research focus will be Z instead of Y_i and to jump to a new state j , the range of Z will be

$$Z \in (\alpha_i(j-1), \alpha_i(j))$$

Step 3: Constructing Jump Transition Rates

• 3.1 Basic Idea

We want to calculate: the intensity $q_{i,j}$ of jumping from state i to state j .

Mathematical Expression: $q_{i,j} = \int_{\text{corresponding } Z \text{ interval}} \lambda(z) \cdot f_Z(z) \, dz$

Plain Language: To calculate the intensity from state i to state j , one direct and intuitive idea is that we will multiply the "whole intensity" by the corresponding probability density function and do the integration over the whole range for the jump size.

• 3.2 Considering State-Dependent Intensity

State-dependent jump intensity reflects the empirical fact that identical news events have different relative impacts on assets with different price levels:

- The same type of news (e.g., regulatory changes, supply disruptions) causes larger **relative price movements** in smaller companies than in larger ones
- A 5% supply shock may cause dramatic price jumps in a small commodity market but relatively modest moves in a large, liquid market
- **Jump frequency and magnitude should scale with the current price level** to reflect this market microstructure reality

Mathematical Foundation

Following Lian and Song (2021), the CEV-DEJD model incorporates state-dependent jump intensity. In their formulation, the jump rates explicitly include the factor $\left(\frac{x_i^h}{F_0}\right)^{\beta}$, where:

- This maintains consistency with the CEV framework's philosophy of state-dependent volatility
- The same elasticity parameter β governs both diffusion and jump components
- This formulation has been empirically validated in commodity futures markets

Mathematical Correction

Under the CEV model framework, jump intensity depends on the current state:

$$\text{Effective jump rate} = \lambda \left(\frac{x_i^h}{F_0} \right)^{\beta}$$

Where:

- λ is the base jump intensity parameter
- F_0 is the reference price level
- β controls the degree of state dependence

• 3.3 Probability density function of Z

Because our research focus is **the relative jump size Z rather than the jump amplitude Y**, we need to investigate about PDF of the new random variable Z.

In the previous setup, we know that **$Z = e^Y - 1$** and the probability density function of Y can be expressed as follows:

$$f_Y(y) = p\eta_1 e^{-\eta_1 y} \mathbf{1}_{y \geq 0} + (1 - p)\eta_2 e^{\eta_2 y} \mathbf{1}_{y < 0}$$

As a consequence, this problem is transformed into one kind of question like that given the distribution of one random variable, what's the distribution of the transformation of this random variable?

The answer to this question is as follows: assuming that we have the distribution of X, then the PDF of $Y = g(X)$ is

$$f_Y(y) = f_X(g^{-1}(y)) \left| \frac{d}{dy} g^{-1}(y) \right|$$

The result for the probability density function of Z is as follows:

$$f_Z(z) = p\eta_1 (z + 1)^{-1-\eta_1} \mathbf{1}_{z \geq 0} + (1 - p)\eta_2 (z + 1)^{\eta_2-1} \mathbf{1}_{(-1,0)}$$

• 3.4 Final Result of $q_{i,j}$:

$$q_{i,j}^J = \int_{\alpha_i(j-1)}^{\alpha_i(j)} \lambda \left(\frac{x_i^h}{F_0} \right)^\beta [p\eta_1 (z + 1)^{-1-\eta_1} \mathbf{1}_{z \geq 0} + (1 - p)\eta_2 (z + 1)^{\eta_2-1} \mathbf{1}_{-1 < z < 0}] dz$$

$$= \begin{cases} 0 & \text{if } i = 1, N, \\ \int_{\alpha_i(j-1)}^{\alpha_i(j)} \lambda \left(\frac{x_i^h}{F_0} \right)^\beta p\eta_1 (z + 1)^{-1-\eta_1} dz & \text{if } i \neq 1, N; i \neq j; \\ & \alpha_i(j-1) \geq 0 \text{ and } \alpha_i(j) \geq 0, \\ \int_{\alpha_i(j-1)}^{\alpha_i(j)} \lambda \left(\frac{x_i^h}{F_0} \right)^\beta (1 - p)\eta_2 (z + 1)^{\eta_2-1} dz & \text{if } i \neq 1, N; i \neq j; \\ & \alpha_i(j-1) < 0 \text{ and } \alpha_i(j) < 0, \\ \int_{\alpha_i(j-1)}^{\alpha_i(j)} \lambda \left(\frac{x_i^h}{F_0} \right)^\beta [p\eta_1 (z + 1)^{-1-\eta_1} \mathbf{1}_{z \geq 0} + (1 - p)\eta_2 (z + 1)^{\eta_2-1} \mathbf{1}_{-1 < z < 0}] dz & \text{if } i \neq 1, N; i \neq j; \\ & \alpha_i(j-1) \cdot \alpha_i(j) < 0, \\ - \sum_{\substack{j=1 \\ j \neq i}}^N q_{i,j}^J & \text{if } i \neq 1, N; i = j. \end{cases}$$

Step 4: Case-by-Case Integration

• 4.1 Case 1: Pure Positive Jump

Condition: $\alpha_i(j-1) \geq 0$ and $\alpha_i(j) \geq 0$

This means that the entire interval for state j lies **above** the current x_i^h (jump upward).

Plain language: We can imagine the following scenario—— the good news allows the price to increase. The minimum of the range of the target state is larger than the current price level, so the jump amplitude should be completely positive to reach the target price state.

Integration Part:

$$q_{i,j}^J = \int_{\alpha_i(j-1)}^{\alpha_i(j)} \lambda \left(\frac{x_i^h}{F_0} \right)^\beta [p\eta_1 (z + 1)^{-1-\eta_1}] dz$$

Calculation Process:

$$\begin{aligned}
q_{i,j}^J &= \lambda \left(\frac{x_i^h}{F_0} \right)^\beta \int_{\alpha_i(j-1)}^{\alpha_i(j)} [p\eta_1(z+1)^{-1-\eta_1}] dz \\
&= \lambda \left(\frac{x_i^h}{F_0} \right)^\beta p\eta_1 \left(-\frac{1}{\eta_1} \right) (1+z)^{-\eta_1} \Big|_{\alpha_i(j-1)}^{\alpha_i(j)} \\
&= -\lambda p \left(\frac{x_i^h}{F_0} \right)^\beta [(\alpha_i(j)+1)^{-\eta_1} - (\alpha_i(j-1)+1)^{-\eta_1}]
\end{aligned}$$

- 4.2 Case 2: Pure Negative Jump

Condition: $\alpha_i(j-1) < 0$ and $\alpha_i(j) < 0$

This means: the entire interval for state j lies **below** the current x_i^h (jump downward).

Plain language: We can imagine the following scenario—the bad news makes the price decrease. The maximum of the range of the target state is smaller than the current price level, so the jump amplitude should be completely negative to reach the target price state.

Integration Part:

$$q_{i,j}^J = \int_{\alpha_i(j-1)}^{\alpha_i(j)} \lambda \left(\frac{x_i^h}{F_0} \right)^\beta (1-p)\eta_2(z+1)^{\eta_2-1} dz$$

Calculation Process

$$\begin{aligned}
q_{i,j}^J &= \lambda \left(\frac{x_i^h}{F_0} \right)^\beta \int_{\alpha_i(j-1)}^{\alpha_i(j)} (1-p)\eta_2(z+1)^{\eta_2-1} dz \\
&= \lambda(1-p) \left(\frac{x_i^h}{F_0} \right)^\beta \eta_2 \frac{1}{\eta_2} (z+1)^{\eta_2} \Big|_{\alpha_i(j-1)}^{\alpha_i(j)} \\
&= \lambda(1-p) \left(\frac{x_i^h}{F_0} \right)^\beta [(\alpha_i(j)+1)^{\eta_2} - (\alpha_i(j-1)+1)^{\eta_2}]
\end{aligned}$$

- 4.3 Case 3: Neutral Jump

Condition: $\alpha_i(j-1) < 0$ and $\alpha_i(j) > 0$

This means: the current x_i^h lies in the entire interval for state j .

Integration Part:

$$q_{i,j}^J = \int_{\alpha_i(j-1)}^{\alpha_i(j)} \lambda \left(\frac{x_i^h}{F_0} \right)^\beta [p\eta_1(z+1)^{-1-\eta_1} + (1-p)\eta_2(z+1)^{\eta_2-1}] dz$$

Calculation Process:

$$\begin{aligned}
q_{i,j}^J &= \lambda \left(\frac{x_i^h}{F_0} \right)^\beta \int_{\alpha_i(j-1)}^{\alpha_i(j)} [p\eta_1(z+1)^{-1-\eta_1} + (1-p)\eta_2(z+1)^{\eta_2-1}] dz \\
&= \lambda \left(\frac{x_i^h}{F_0} \right)^\beta \left[\int_{\alpha_i(j-1)}^0 (1-p)\eta_2(z+1)^{\eta_2-1} dz + \int_0^{\alpha_i(j)} p\eta_1(z+1)^{-1-\eta_1} dz \right] \\
&= \lambda \left(\frac{x_i^h}{F_0} \right)^\beta [(1-p)(z+1)^{\eta_2} \Big|_{\alpha_i(j-1)}^0 - p(1+z)^{-\eta_1} \Big|_0^{\alpha_i(j)}] \\
&= \lambda \left(\frac{x_i^h}{F_0} \right)^\beta \{ (1-p)[1 - (\alpha_i(j-1)+1)^{\eta_2}] - p[(\alpha_i(j)+1)^{-\eta_1} - 1] \}
\end{aligned}$$

Part B: Constructing the Diffusion Component Q^D

In the construction of the matrix of the pure diffusion, we also assume that the non-zero elements are only at the upper diagonal, lower diagonal and the diagonal.

One critical rule for this construction is that this part should act as the compensation for the jump matrix to jointly form the two moments of the jump diffusion process.

The Role of Q^D : Diffusion as Compensation

In jump-diffusion models, Q^D serves a **dual purpose**:

1. **Capture Continuous Movements**: Model normal price fluctuations via diffusion
2. **Compensate Jump Moments**: Adjust diffusion rates to ensure the **combined process** ($Q^D + Q^J$) has the correct risk-neutral drift (zero) and total variance.

Why Compensation is Needed:

The jump component Q^J alone introduces an **expected drift** from asymmetric jumps and contributes to total variance. Under the risk-neutral measure, the futures price must be a **martingale** (expected drift = 0). Therefore, Q^D must be adjusted to:

- **Offset** the expected drift from jumps
- **Complement** the variance contribution from jumps to match the model's total instantaneous variance

Think of it as a **budget allocation problem**:

- **Total Variance Target** = $\sigma^2(F_t/F_0)^{2\beta} + \text{Jump Variance}$
- **Total Drift Target** = 0 (martingale condition)
- Q^D is solved to meet these targets *after* Q^J is fixed.

The first moment of this process— $\mathbb{E}[dF_t | F_t]$ is relatively intuitive according to the idea that under the risk-neutral measure, all the assets should be a martingale, whose expectation of the corresponding change should be 0.

However, the second moment $\frac{\mathbb{E}[(dF_t)^2 | F_t]}{dt}$ is very difficult to calculate. Here, I will provide the detailed calculation process:

We start with the basic form of $(dF_t)^2$:

$$(dF_t)^2 = F_t^2(-\lambda \zeta dt + \sigma \left(\frac{F_t}{F_0}\right)^\beta dW_t + d\left(\sum_{i=1}^{N_t}(e^{Y_i} - 1)\right))^2$$

Then, we expand the square terms and get the simplified result via quadratic variance and the independence between the jump and the diffusion part:

$$(dF_t)^2 = F_t^2[\sigma^2 \left(\frac{F_t}{F_0}\right)^{2\beta} (dW_t)^2 + (d\left(\sum_{i=1}^{N_t}(e^{Y_i} - 1)\right))^2]$$

Take expectations on both sides of the former equation:

$$\mathbb{E}[(dF_t)^2 | F_t] = F_t^2 \sigma^2 \left(\frac{F_t}{F_0}\right)^{2\beta} dt + \lambda dt \cdot \mathbb{E}[(e^{Y_i} - 1)^2]$$

Then, we can focus on the part $\mathbb{E}[(e^{Y_i} - 1)^2]$

$$\mathbb{E}[(e^{Y_i} - 1)^2] = \mathbb{E}[e^{2Y_i} - 2e^{Y_i} + 1]$$

For the double-exponential Random Variable, the moment generating function(mgf) is as follows:

$$\mathbb{E}[e^{kY_i}] = p \frac{\eta_1}{\eta_1 - k} + (1 - p) \frac{\eta_2}{\eta_2 + k} \quad (\text{for } k < \eta_1)$$

According to the mgf, we can simplify $\mathbb{E}[(e^{Y_i} - 1)^2]$

$$\begin{aligned} \mathbb{E}[(e^{Y_i} - 1)^2] &= p \frac{\eta_1}{\eta_1 - 2} + (1 - p) \frac{\eta_2}{\eta_2 + 2} - 2p \frac{\eta_1}{\eta_1 - 1} - 2(1 - p) \frac{\eta_2}{\eta_2 + 1} + 1 \\ &= 2 \left(\frac{p}{(\eta_1 - 1)(\eta_1 - 2)} + \frac{1 - p}{(\eta_2 + 1)(\eta_2 + 2)} \right) \end{aligned}$$

Therefore, the second moment

$$\frac{\mathbb{E}[(dF_t)^2 | F_t]}{dt} = F_t^2 \left[\sigma^2 \left(\frac{F_t}{F_0} \right)^{2\beta} + \left(\frac{F_t}{F_0} \right)^\beta 2\lambda \left(\frac{p}{(\eta_1 - 1)(\eta_1 - 2)} + \frac{1 - p}{(\eta_2 + 1)(\eta_2 + 2)} \right) \right]$$

Then, we denote the targets $\Lambda^{D\{i,i-1\}}$, $\Lambda^{D\{i,i+1\}}$ and $\Lambda^{D_{-i,i}}$ as d_i^- , d_i^+ and d_i^0 and start the calculation of the transition rate matrix of the diffusion process

We have three equations, which are written as follows:

1. The summation of the elements in one row is equal to 0: $d_i^- + d_i^+ + d_i^0 = 0$.
2. The first moment is equal to 0 (under the risk-neutral measure): $\sum_{j=1}^N \Lambda^{D\{ij\}} (x_j - x_i) + \sum_{j=1}^N \Lambda^{D_{-i,j}} (x_j - x_i) = 0$
3. The second moment is equal to the value calculated above:

$$\sum_{j=1}^N \Lambda_{ij}^D (x_j - x_i)^2 + \sum_{j=1}^N \Lambda_{ij}^J (x_j - x_i)^2 = F_t^2 \left[\sigma^2 \left(\frac{F_t}{F_0} \right)^{2\beta} + \left(\frac{F_t}{F_0} \right)^\beta 2\lambda \left(\frac{p}{(\eta_1 - 1)(\eta_1 - 2)} + \frac{1 - p}{(\eta_2 + 1)(\eta_2 + 2)} \right) \right]$$

According to the three conditions above, we can get the formula for the matrix of the diffusion part:

$$\begin{cases} d_i^- + d_i^+ + d_i^0 = 0 & \Rightarrow d_i^0 = -(d_i^- + d_i^+) \\ d_i^- \cdot (-h_i) + d_i^+ \cdot (h_i) = -\sum_{j=1}^N \Lambda_{ij}^J (x_j - x_i) \\ (d_i^- + d_i^+) h_i^2 = F_t^2 \left[\sigma^2 \left(\frac{F_t}{F_0} \right)^{2\beta} + \left(\frac{F_t}{F_0} \right)^\beta 2\lambda \left(\frac{p}{(\eta_1 - 1)(\eta_1 - 2)} + \frac{1 - p}{(\eta_2 + 1)(\eta_2 + 2)} \right) \right] \end{cases}$$

The solution is as follows:

$$\begin{cases} d_i^+ = \frac{F_t^2 \left[\sigma^2 \left(\frac{F_t}{F_0} \right)^{2\beta} + \left(\frac{F_t}{F_0} \right)^\beta 2\lambda \left(\frac{p}{(\eta_1 - 1)(\eta_1 - 2)} + \frac{1 - p}{(\eta_2 + 1)(\eta_2 + 2)} \right) \right] - h_i \sum_{j=1}^N \Lambda_{ij}^J (x_j - x_i)}{h_i^2} \\ d_i^- = \frac{F_t^2 \left[\sigma^2 \left(\frac{F_t}{F_0} \right)^{2\beta} + \left(\frac{F_t}{F_0} \right)^\beta 2\lambda \left(\frac{p}{(\eta_1 - 1)(\eta_1 - 2)} + \frac{1 - p}{(\eta_2 + 1)(\eta_2 + 2)} \right) \right] + h_i \sum_{j=1}^N \Lambda_{ij}^J (x_j - x_i)}{h_i^2} \\ d_i^0 = -(d_i^- + d_i^+) \end{cases}$$

Algorithm: Constructing $Q^A D$ given $Q^A J$ and Model Parameters

For each interior state i (with neighboring states $i-1$ and $i+1$):

1. Compute Jump Moment Contributions:

- Net Jump Drift: $\text{Drift} = \sum_{j=1}^N (x_j - x_i) \backslash, q_{\{ij\}}^A$
- Jump Variance: $\text{Var} = \sum_{j=1}^N (x_j - x_i)^2 \backslash, q_{\{ij\}}^A$

2. Compute Total Variance Target from Model:

$$\text{Var}_{\text{total}} = (x_i^h)^2 \left[\sigma^2 \left(\frac{x_i^h}{F_0} \right)^{2\beta} + \left(\frac{x_i^h}{F_0} \right)^\beta 2\lambda \left(\frac{p}{(\eta_1 - 1)(\eta_1 - 2)} + \frac{1 - p}{(\eta_2 + 1)(\eta_2 + 2)} \right) \right]$$

3. Solve for Diffusion Rates:

Let $h_i = x_{i+1}^h - x_i^h$.

- $d_i^+ = \frac{\text{Var}\{\text{total}\}}{\text{total}} - h_i \cdot \text{Drift}\{h_i^2\}$
- $d_i^- = \frac{\text{Var}\{\text{total}\}}{\text{total}} + h_i \cdot \text{Drift}\{h_i^2\}$
- $d_i^0 = -(d_i^+ + d_i^-)$

4. Assign to Q^D :

- $q_{i,i+1}^D = d_i^+$
- $q_{i,i-1}^D = d_i^-$
- $q_{i,i}^D = d_i^0$

Boundary States ($i=1$ and $i=N$): require special treatment (e.g., reflecting or absorbing conditions) which modifies the moment matching equations. A common approach is to use a *truncated* grid and adjust the jump probabilities at boundaries accordingly.

Part C: Assembling and Validating the Complete Matrix Q

Complete Generator Matrix Q Visualization:

For example, the complete generator matrix $Q = Q^D + Q^J$ for a 5-state system has the following structure:

$$Q = \begin{pmatrix} q_{11} & q_{12} & q_{13} & q_{14} & q_{15} \\ q_{21} & q_{22} & q_{23} & q_{24} & q_{25} \\ q_{31} & q_{32} & q_{33} & q_{34} & q_{35} \\ q_{41} & q_{42} & q_{43} & q_{44} & q_{45} \\ q_{51} & q_{52} & q_{53} & q_{54} & q_{55} \end{pmatrix}$$

Where the elements are:

- **Tridiagonal elements** ($|i-j| \leq 1$): $q_{i,j} = q_{i,j}^D + q_{i,j}^J$ (both diffusion and jump contributions)
- **Off-tridiagonal elements** ($|i-j| > 1$): $q_{i,j} = q_{i,j}^J$ (jump contributions only)
- **Diagonal elements:** $q_{i,i} = -\sum_{j \neq i} q_{i,j}$ (negative sum of row)

Specifically for a 5-state system:

$$Q = \begin{pmatrix} q_{11} & q_{12} & q_{13} & q_{14} & q_{15} \\ q_{21} & q_{22} & q_{23} & q_{24} & q_{25} \\ q_{31} & q_{32} & q_{33} & q_{34} & q_{35} \\ q_{41} & q_{42} & q_{43} & q_{44} & q_{45} \\ q_{51} & q_{52} & q_{53} & q_{54} & q_{55} \end{pmatrix} = \begin{pmatrix} -\sum_{j=2}^5 q_{1j} & q_{12}^D + q_{12}^J & q_{13}^J & q_{14}^J & q_{15}^J \\ q_{21}^D + q_{21}^J & -\sum_{j \neq 2} q_{2j} & q_{23}^D + q_{23}^J & q_{24}^J & q_{25}^J \\ q_{31}^J & q_{32}^D + q_{32}^J & -\sum_{j \neq 3} q_{3j} & q_{34}^D + q_{34}^J & q_{35}^J \\ q_{41}^J & q_{42}^J & q_{43}^D + q_{43}^J & -\sum_{j \neq 4} q_{4j} & q_{45}^D + q_{45}^J \\ q_{51}^J & q_{52}^J & q_{53}^J & q_{54}^D + q_{54}^J & -\sum_{j=1}^4 q_{5j} \end{pmatrix}$$

Key Properties of Q :

1. **Row sum zero:** $\sum_{j=1}^N q_{ij} = 0$ for all i
2. **Non-negative off-diagonals:** $q_{i,j} \geq 0$ for $i \neq j$
3. **Negative diagonals:** $q_{i,i} < 0$ (except in degenerate cases)
4. **Sparsity:** Tridiagonal structure for diffusion plus additional non-zero entries from jumps

Numerical Example: Assembling Q for a 3-State System

Consider a minimal example with $N=3$ price states: $[90, 100, 110]$ and $F_0 = 100$.

1. Parameter Values (Illustrative):

- $\sigma = 0.2$, $\beta = -0.5$, $\lambda = 5$, $p = 0.6$, $\eta_1 = 10$, $\eta_2 = 15$
- Risk-free rate $r = 0.03$, no dividends

2. Compute Q^J (Jump Component):

For state $i=2$ (price = 100):

- Jump to state 1 ($j=1$): Negative jump needed. Calculate q_{21}^J using **Case 2** integration.
- Jump to state 3 ($j=3$): Positive jump needed. Calculate q_{23}^J using **Case 1** integration.
(Detailed calculation follows the formulas in Section 3.4)

Example result (illustrative rates per year):

$$Q^J \approx \begin{pmatrix} -2.5 & 0 & 2.5 \\ 1.2 & -3.0 & 1.8 \\ 0.5 & 0 & -0.5 \end{pmatrix}$$

3. Compute Q^D (Diffusion Component with Compensation):

Using the moment matching equations:

- Calculate expected jump drift from Q^J : $\sum_j (x_j - x_i) q_{ij}^J$
- Calculate total variance target from model SDE
- Solve for d_i^+ , d_i^- to satisfy both moment conditions

Example result:

$$Q^D \approx \begin{pmatrix} -0.12 & 0.12 & 0 \\ 0.08 & -0.16 & 0.08 \\ 0 & 0.10 & -0.10 \end{pmatrix}$$

4. Combine into Final Q :

$$Q = Q^D + Q^J = \begin{pmatrix} -2.62 & 0.12 & 2.50 \\ 1.28 & -3.16 & 1.88 \\ 0.50 & 0.10 & -0.60 \end{pmatrix}$$

5. Verification:

- Row sums: $[0, 0, 0]$ ✓ (martingale, up to numerical precision)
- All off-diagonals ≥ 0 ✓
- Diagonal < 0 ✓

This small example demonstrates how the **compensation mechanism** in Q^D ensures the combined generator produces a valid risk-neutral CTMC.

Parameter Estimation

Parameter estimation for the CEV-DEJD model via CTMC approximation follows the Maximum Likelihood Estimation (MLE) framework, but requires careful handling of the jump component and the dense generator matrix Q .

Likelihood Function Construction

Given a sequence of discretized price observations $\{F_{t_1}, F_{t_2}, \dots, F_{t_n}\}$ mapped to state indices $\{s_1, s_2, \dots, s_n\}$, the likelihood function is:

$$L(\theta) = \pi(s_1) \prod_{k=1}^{n-1} P_{\Delta t}(\theta)_{s_k, s_{k+1}}$$

where:

- $\theta = (\sigma, \beta, \lambda, p, \eta_1, \eta_2)$ is the parameter vector
- $\pi(s_1)$ is the initial state distribution (often uniform or stationary)
- $P_{\Delta t}(\theta) = \exp(\Delta t \cdot Q(\theta))$ is the transition probability matrix
- Δt is the time interval between observations (e.g., 1/252 for daily data)

Log-Likelihood and Optimization

The log-likelihood is:

$$\ell(\theta) = \log \pi(s_1) + \sum_{k=1}^{n-1} \log[P_{\Delta t}(\theta)_{s_k, s_{k+1}}]$$

The MLE estimator is obtained by:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \ell(\theta)$$

subject to constraints: $\sigma > 0$, $\lambda > 0$, $0 \leq p \leq 1$, $\eta_1 > 1$, $\eta_2 > 0$.

Special Considerations for CEV-DEJD

1. Computational Challenges:

- $Q(\theta)$ is **not sparse** due to $Q^{\wedge}$ component $\rightarrow$ matrix exponential $e^{\Delta t Q}$ is computationally expensive
- Consider using **uniformization (randomization) method**:

$$P_{\Delta t} = e^{\Delta t Q} = \sum_{m=0}^{\infty} e^{-\Lambda \Delta t} \frac{(\Lambda \Delta t)^m}{m!} S^m$$

where $\Lambda = \max_i |q_{ii}|$ and $S = I + Q/\Lambda$

2. Initialization Strategy:

- **Step 1:** Estimate pure CEV model (σ, β) ignoring jumps
- **Step 2:** Analyze residuals to estimate jump parameters via method of moments:
 - $\hat{\lambda}$ from frequency of large returns
 - $\hat{p}$ from proportion of positive vs negative large jumps
 - $\hat{\eta}_1, \hat{\eta}_2$ from tail decay rates
- **Step 3:** Use these as initial values for full MLE optimization

3. Identification Issues:

- The parameters σ (diffusion volatility) and λ (jump intensity) both contribute to total variance. High-frequency data helps separate them.
- For state-dependent intensity $\lambda(F_t) = \lambda(F_0)^{\beta}$, the same β affects both diffusion and jump components, requiring joint estimation.

4. Numerical Optimization Tips:

- Use **gradient-free methods** (Nelder-Mead, Powell) initially due to complex likelihood surface
- For gradient-based methods, compute derivatives via finite differences
- Implement **parameter constraints** via transformations (e.g., $\eta_1 = 1 + e^{\tilde{\eta}_1}$ to ensure $\eta_1 > 1$)

Validation of Estimates

After obtaining $\hat{\theta}$, validate by:

1. **In-sample fit:** Compare empirical vs model-implied transition probabilities
2. **Moment matching:** Check if the estimated Q reproduces sample moments (mean, variance, skewness, kurtosis)
3. **Residual analysis:** Examine standardized residuals for remaining structure

Pricing Process

Pricing American options under the CEV-DEJD model using CTMC approximation follows the **backward dynamic programming** framework, with adaptations for the jump component and state-dependent volatility.

Backward Induction Algorithm

Time Discretization:

Divide $[0, T]$ into L steps: $0 = t_0 < t_1 < \dots < t_L = T$, with $\Delta t = T/L$. American options are approximated as Bermudan options exercisable at these discrete times.

State Value Matrix:

Define $V \in \mathbb{R}^{N \times (L+1)}$ where $V(i, \ell)$ is the option value at time t_ℓ when the CTMC is in state i (price = x_i^h).

Terminal Condition ($\ell = L$, maturity T):

- Put option: $V(i, L) = \max(K - x_i^h, 0)$
- Call option: $V(i, L) = \max(x_i^h - K, 0)$

Backward Recursion ($\ell = L-1, L-2, \dots, 0$):

For each state i :

1. **Continuation value:**

$$C(i, \ell) = e^{-r\Delta t} \sum_{j=1}^N P_{\Delta t}(\hat{\theta})_{i,j} \cdot V(j, \ell + 1)$$

where $P_{\Delta t}(\hat{\theta}) = \exp(\Delta t \cdot Q(\hat{\theta}))$ uses the estimated parameters.

2. **Immediate exercise value:**

$$E(i, \ell) = \max(K - x_i^h, 0) \quad \text{put}$$

3. **American option value:**

$$V(i, \ell) = \max(E(i, \ell), C(i, \ell))$$

Special Considerations for CEV-DEJD

1. Grid Design:

- Use **non-uniform grid** denser near strike price K and current spot F_0
- Ensure grid covers ± 5 standard deviations of jump distribution
- For $\beta < 0$ (commodities), place more points at lower price levels where volatility is higher

2. Early Exercise Boundary:

The jump component affects optimal exercise timing:

- **Downward jumps** make early exercise more attractive for puts (insurance against crashes)
- **Upward jumps** make early exercise less attractive for calls (opportunity cost)
- Compute exercise boundary $B(t_{\ell})$ as the highest price where $E(i, \ell) > C(i, \ell)$

3. Matrix Exponential Computation:

- For pricing, Δt is much smaller than for estimation (e.g., $T/1000$ vs $1/252$)
- Use **scaling and squaring** or **Padé approximation** for accurate $e^{\Delta t Q}$
- Consider **Krylov subspace methods** for large N (e.g., $N > 500$)

4. Risk-Neutral Adjustment:

Ensure the estimated parameters $\hat{\theta}$ are under the **risk-neutral measure** $\mathbb{Q}$:

- If estimated from historical data, adjust μ (drift) to $r - q$ (risk-free rate minus dividend yield)
- For jumps, risk-neutral parameters may differ from physical ones (need risk premia adjustment)

Final Price Computation

Given current futures price F_0 :

1. Find nearest state: $i_0 = \arg\min_i |x_i^h - F_0|$
2. Option price: $V_0 = V(i_0, 0)$
3. For smoother results, use interpolation:

$$V_0 = \sum_{j=1}^N w_j \cdot V(j, 1)$$

where weights w_j are based on distance from F_0 to grid points.

Example 3

Model Specification

The Regime-Switching CEV (RS-CEV) model is a two-dimensional Markov process $\{(R_t, F_t)\}_{t \geq 0}$, where:

- $F_t > 0$ is the observable futures price
- $R_t \in \{1, 2, \dots, M\}$ is an unobservable (hidden) market regime, typically $M = 2$

Price Dynamics (Conditional on Regime):

Under the risk-neutral measure $\mathbb{Q}$, given $R_t = i$, the futures price follows:

$$\frac{dF_t}{F_t} = \sigma_i \left(\frac{F_t}{F_0} \right)^{\beta_i} dW_t, \quad i = 1, \dots, M \quad (\text{RS.1})$$

Parameter Meanings:

- $\sigma_i > 0$: Baseline volatility parameter in regime i (controls overall fluctuation level)
- β_i : Elasticity parameter in regime i
 - $\beta_i < 0$: Inverse leverage effect (volatility $\uparrow$ when price $\downarrow$) – common in commodities
 - $\beta_i = 0$: Geometric Brownian Motion (constant volatility)
 - $\beta_i > 0$: Leverage effect (volatility $\downarrow$ when price $\downarrow$) – typical in stock markets
- W_t : Standard Brownian motion driving price fluctuations

Regime Switching Process:

R_t is a Continuous-Time Markov Chain (CTMC) with generator matrix Q^R :

$$Q^R = \begin{pmatrix} -\lambda_1 & \lambda_{12} & \cdots & \lambda_{1M} \\ \lambda_{21} & -\lambda_2 & \cdots & \lambda_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ \lambda_{M1} & \lambda_{M2} & \cdots & -\lambda_M \end{pmatrix} \quad (\text{RS.2})$$

For $M=2$:

$$Q^R = \begin{pmatrix} -\lambda_1 & \lambda_{12} \\ \lambda_{21} & -\lambda_2 \end{pmatrix} \quad (\text{RS.3})$$

- λ_{12} : Rate of switching from Regime 1 to Regime 2 (events per unit time)
- λ_{21} : Rate of switching from Regime 2 to Regime 1
- Average sojourn time in regime i : $1/\lambda_i$ where $\lambda_1 = \lambda_{12}$, $\lambda_2 = \lambda_{21}$
- Stationary distribution: $\pi_1 = \frac{\lambda_{21}}{\lambda_{12} + \lambda_{21}}$, $\pi_2 = \frac{\lambda_{12}}{\lambda_{12} + \lambda_{21}}$

Key Assumptions:

1. **Conditional diffusion assumption:** Given the regime path, price evolution depends only on current regime, and W_t is continuous across regime switches (no price jumps from switching)
2. **Independence:** Regime switching drivers are independent of W_t

Approximation Process

The RS-CEV model is approximated by a **single CTMC** with state space combining both price and regime dimensions.

State Space Setup

Combined State Space:

$$\mathcal{S} = \{(x_n, m) \mid n = 1, \dots, N; m = 1, \dots, M\} \quad (\text{RS.4})$$

Total states: $K = M \times N$.

State index mapping: $k = (m-1) \times N + n$ corresponds to price at grid point x_n and market in regime m .

Constructing the Generator Matrix Q

The generator Q is a $K \times K$ block matrix:

$$Q = \begin{pmatrix} Q^{11} & Q^{12} & \dots & Q^{1M} \\ Q^{21} & Q^{22} & \dots & Q^{2M} \\ \vdots & \vdots & \ddots & \vdots \\ Q^{M1} & Q^{M2} & \dots & Q^{MM} \end{pmatrix} \quad (\text{RS.5})$$

Step 1: Diffusion Blocks (Within Each Regime)

For each regime $m = 1, \dots, M$, construct an $N \times N$ tridiagonal matrix $\Lambda^{(m)}$ using the CEV diffusion formula.

First, derive the continuous infinitesimal generator for regime m based on the SDE: $\frac{dF_t}{F_t} = \sigma_m \left(\frac{F_t}{F_0} \right)^{\beta_m} dW_t$

Apply Itô's lemma to $X_t = \ln F_t$:

$$dX_t = -\frac{1}{2} \sigma_m^2 \left(\frac{F_t}{F_0} \right)^{2\beta_m} dt + \sigma_m \left(\frac{F_t}{F_0} \right)^{\beta_m} dW_t$$

The infinitesimal generator acting on test function f is:

$$\mathcal{G}^{(m)} f(x) = \frac{1}{2} \sigma_m^2 x^2 \left(\frac{x}{F_0} \right)^{2\beta_m} f''(x) \quad (\text{RS.6})$$

Now construct the discrete CTMC generator $\Lambda^{(m)}$ that converges to $\mathcal{G}^{(m)}$.

For interior grid points $1 < n < N$, define:

- $h_n = x_{n+1} - x_n$
- $h_{n-1} = x_n - x_{n-1}$
- $D_n = \sigma_m^2 x_n^{2\beta_m+2} / F_0^{2\beta_m}$

Through **moment matching** (detailed Taylor expansion):

1. **Property of Generator:** $\Lambda^{(m)}_{n,n-1} + \Lambda^{(m)}_{n,n} + \Lambda^{(m)}_{n,n+1} = 0$
2. **First moment matching (drift):** $-h_{n-1} \Lambda^{(m)}_{n,n-1} + h_n \Lambda^{(m)}_{n,n+1} = 0$
3. **Second moment matching (diffusion):** $h_{n-1}^2 \Lambda^{(m)}_{n,n-1} + h_n^2 \Lambda^{(m)}_{n,n+1} = D_n$

Solving this system yields:

$$\begin{aligned} \Lambda_{n,n-1}^{(m)} &= \frac{D_n}{h_{n-1}(h_n + h_{n-1})} \\ \Lambda_{n,n+1}^{(m)} &= \frac{D_n}{h_n(h_n + h_{n-1})} \\ \Lambda_{n,n}^{(m)} &= -(\Lambda_{n,n-1}^{(m)} + \Lambda_{n,n+1}^{(m)}) = -\frac{D_n}{h_{n-1}h_n} \end{aligned} \quad (\text{RS.7})$$

Boundary points ($n=1$ and $n=N$): Use reflecting boundary conditions:

- For $n=1$: $\Lambda^{(m)}_{1,2} = \frac{D_1}{h_1^2}$, $\Lambda^{(m)}_{1,1} = -\Lambda^{(m)}_{1,2}$

- For $n=N$: $\Lambda^{(m)}_{\{N,N-1\}} = \frac{D_N \{h_{\{N-1\}}^2\}}{\Lambda^{(m)}_{\{N,N\}}} = -\Lambda^{(m)}_{\{N,N-1\}}$

Step 2: Regime Switching Blocks (Between Regimes)

For $m_1 \neq m_2$:

$$Q^{m_1 m_2} = \lambda_{m_1 m_2} I_N \quad (\text{RS.8})$$

where I_N is the $N \times N$ identity matrix.

This means: from any price node x_n in regime m_1 , the system can switch to **the same price node** x_n in regime m_2 at rate $\lambda_{m_1 m_2}$.

Step 3: Diagonal Correction (Within Same Regime)

For $m_1 = m_2 = m$:

$$Q^{mm} = \Lambda^{(m)} - \left(\sum_{j \neq m} \lambda_{mj} \right) I_N \quad (\text{RS.9})$$

The diagonal subtraction ensures each row sums to zero (accounting for probability flow to other regimes).

Example: Two-Regime Case ($M=2$)

$$Q = \begin{pmatrix} \Lambda^{(1)} - \lambda_{11} I_N & \lambda_{12} I_N \\ \lambda_{21} I_N & \Lambda^{(2)} - \lambda_{22} I_N \end{pmatrix} \quad (\text{RS.10})$$

Step 4: Convergence Guarantee

According to FYP Chapter 5 error analysis:

$$\|e^{Gt} f - e^{Qt} f\| \leq t \cdot C \cdot \max_i \|\mathcal{G}^{(i)} f - \Lambda^{(i)} f\| + O(\lambda_{ij} \Delta t) \quad (\text{RS.11})$$

where $\mathcal{G}$ is the continuous generator of the RS process, and C is a constant. Since each $\Lambda^{(i)}$ is constructed via moment matching with $O(h^2)$ error, the overall approximation converges as grid size $h \rightarrow 0$.

Parameter Estimation

The model parameters $\theta = (\sigma_1, \beta_1, \sigma_2, \beta_2, \lambda_{12}, \lambda_{21})$ are estimated via **Maximum Likelihood Estimation (MLE)** with the **Hamilton filter**.

State-Dependent Observation Density

Given F_t and $R_{t+\Delta t} = j$, the conditional distribution of $F_{t+\Delta t}$ is approximately log-normal.

Define instantaneous volatility function:

$$\nu_j(F_t) = \sigma_j F_t \left(\frac{F_t}{F_0} \right)^{\beta_j} = \sigma_j F_0^{-\beta_j} F_t^{1+\beta_j} \quad (\text{RS.12})$$

Apply Itô's lemma to $\ln F_t$ and discretize:

$$\ln F_{t+\Delta t} = \ln F_t - \frac{1}{2} \nu_j^2(F_t) \Delta t + \nu_j(F_t) \sqrt{\Delta t} \epsilon_{t+\Delta t}, \quad \epsilon \sim \mathcal{N}(0, 1) \quad (\text{RS.13})$$

Thus:

$$F_{t+\Delta t} \mid (F_t, R_{t+\Delta t} = j) \sim \text{Lognormal}(\mu_j, \sigma_j^2)$$

where:

$$\mu_j = \ln F_t - \frac{1}{2} \nu_j^2(F_t) \Delta t, \quad \sigma_j^2 = \nu_j^2(F_t) \Delta t \quad (\text{RS.14})$$

The state-dependent observation density is:

$$f(F_{t+\Delta t} \mid F_t, R_{t+\Delta t} = j; \theta) = \frac{1}{F_{t+\Delta t} \sqrt{2\pi \nu_j^2(F_t) \Delta t}} \times \exp \left(- \frac{\left[\ln \left(\frac{F_{t+\Delta t}}{F_t} \right) + \frac{1}{2} \nu_j^2(F_t) \Delta t \right]^2}{2 \nu_j^2(F_t) \Delta t} \right) \quad (\text{RS.15})$$

Hamilton Filter Recursion

Let $\xi_{t|i}(i) = \mathbb{P}(R_t = i \mid \mathcal{F}_t)$ be the filtered probability.

1. Prediction step:

$$\xi_{t+\Delta t|t}(j) = \sum_{i=1}^M \xi_t(i) P_{ij}^R(\Delta t), \quad j = 1, \dots, M \quad (\text{RS.16})$$

where $P^R(\Delta t) = \exp(Q^R \Delta t)$ is the regime transition matrix.

2. Update step (Bayes' theorem):

$$\xi_{t+\Delta t}(j) = \frac{\xi_{t+\Delta t|t}(j) \cdot f(F_{t+\Delta t} \mid F_t, R_{t+\Delta t} = j; \theta)}{\sum_{k=1}^M \xi_{t+\Delta t|t}(k) \cdot f(F_{t+\Delta t} \mid F_t, R_{t+\Delta t} = k; \theta)} \quad (\text{RS.17})$$

3. Likelihood contribution:

$$L_{t+\Delta t} = \sum_{j=1}^M \xi_{t+\Delta t|t}(j) \cdot f(F_{t+\Delta t} \mid F_t, R_{t+\Delta t} = j; \theta) \quad (\text{RS.18})$$

Complete Log-Likelihood Function

For observation sequence $\{F_0, F_1, \dots, F_T\}$:

$$\ell(\theta) = \sum_{t=0}^{T-1} \log \left(\sum_{j=1}^M \xi_{t+\Delta t|t}(j) \cdot f(F_{t+\Delta t} \mid F_t, R_{t+\Delta t} = j; \theta) \right) \quad (\text{RS.19})$$

Initialization: $\xi_{0|i}(i) = \pi_i$ (stationary distribution) or uniform.

Optimization: Use numerical methods (BFGS, Nelder-Mead) to maximize $\ell(\theta)$.

Analytical expression for $P^R(\Delta t)$ when $M=2$:

$$P^R(\Delta t) = \frac{1}{\lambda_{12} + \lambda_{21}} \begin{pmatrix} \lambda_{21} + \lambda_{12} e^{-(\lambda_{12} + \lambda_{21}) \Delta t} & \lambda_{12} (1 - e^{-(\lambda_{12} + \lambda_{21}) \Delta t}) \\ \lambda_{21} (1 - e^{-(\lambda_{12} + \lambda_{21}) \Delta t}) & \lambda_{12} + \lambda_{21} e^{-(\lambda_{12} + \lambda_{21}) \Delta t} \end{pmatrix} \quad (\text{RS.20})$$

Proof:

We consider a two-state continuous-time Markov chain with regimes 1 and 2. The infinitesimal generator (transition rate matrix) is:

$$Q^R = \begin{pmatrix} -\lambda_{12} & \lambda_{12} \\ \lambda_{21} & -\lambda_{21} \end{pmatrix}$$

where $\lambda_{12} > 0$ is the rate from regime 1 to regime 2, and $\lambda_{21} > 0$ is the rate from regime 2 to regime 1.

We want to compute the transition probability matrix at time Δt :

$$P^R(\Delta t) = \exp(Q^R \Delta t)$$

Step 1:

The transition probabilities $p_{ij}(t) = P(R_t = j \mid R_0 = i)$ satisfy the forward equations:

$$\frac{d}{dt} p_{ij}(t) = \sum_k p_{ik}(t) Q_{kj}^R$$

Step 2:

Let $p_{11}(t) = P(R_t = 1 \mid R_0 = 1)$. The differential equation is:

$$\frac{d}{dt} p_{11}(t) = -\lambda_{12} p_{11}(t) + \lambda_{21} (1 - p_{11}(t))$$

Rearrange:

$$\frac{d}{dt} p_{11}(t) = -(\lambda_{12} + \lambda_{21}) p_{11}(t) + \lambda_{21}$$

This is a first-order linear ODE. The homogeneous solution is:

$$p_h(t) = A e^{-(\lambda_{12} + \lambda_{21})t}$$

Particular solution (constant):

$$p_p(t) = \frac{\lambda_{21}}{\lambda_{12} + \lambda_{21}}$$

General solution:

$$p_{11}(t) = A e^{-(\lambda_{12} + \lambda_{21})t} + \frac{\lambda_{21}}{\lambda_{12} + \lambda_{21}}$$

Initial condition $p_{11}(0) = 1$:

$$A + \frac{\lambda_{21}}{\lambda_{12} + \lambda_{21}} = 1 \quad \Rightarrow \quad A = \frac{\lambda_{12}}{\lambda_{12} + \lambda_{21}}$$

Thus:

$$p_{11}(t) = \frac{\lambda_{21} + \lambda_{12} e^{-(\lambda_{12} + \lambda_{21})t}}{\lambda_{12} + \lambda_{21}}$$

Step 3:

Since $p_{12}(t) = 1 - p_{11}(t)$:

$$p_{12}(t) = \frac{\lambda_{12} (1 - e^{-(\lambda_{12} + \lambda_{21})t})}{\lambda_{12} + \lambda_{21}}$$

By symmetry (or repeating the same procedure starting from regime 2):

$$\begin{aligned} p_{21}(t) &= \frac{\lambda_{21} (1 - e^{-(\lambda_{12} + \lambda_{21})t})}{\lambda_{12} + \lambda_{21}} \\ p_{22}(t) &= \frac{\lambda_{12} + \lambda_{21} e^{-(\lambda_{12} + \lambda_{21})t}}{\lambda_{12} + \lambda_{21}} \end{aligned}$$

Step 4:

Putting all elements together, we obtain:

$$P^R(\Delta t) = \frac{1}{\lambda_{12} + \lambda_{21}} \begin{pmatrix} \lambda_{21} + \lambda_{12} e^{-(\lambda_{12} + \lambda_{21})\Delta t} & \lambda_{12} (1 - e^{-(\lambda_{12} + \lambda_{21})\Delta t}) \\ \lambda_{21} (1 - e^{-(\lambda_{12} + \lambda_{21})\Delta t}) & \lambda_{12} + \lambda_{21} e^{-(\lambda_{12} + \lambda_{21})\Delta t} \end{pmatrix}$$

This is exactly the expression given in equation (RS.20).

Pricing Process

American option pricing under RS-CEV follows **backward dynamic programming** on the combined CTMC.

Time Discretization and Bermudan Approximation

Divide $[0, T]$ into L steps: $\Delta t = T/L$, allow exercise only at $t_\ell = \ell \Delta t$.

State Value Matrix

Define $V \in \mathbb{R}^{K \times (L+1)}$ where:

$$V(k, \ell) = \text{Option value at time } t_\ell \text{ when CTMC is in state } k \quad (\text{RS.21})$$

Terminal condition ($\ell = L$, maturity):

$$V(k, L) = \max(K - x_n, 0), \quad \forall k = (n, m) \in S \quad (\text{RS.22})$$

Backward Induction

For $\ell = L-1, L-2, \dots, 0$:

1. Continuation value:

$$C(k, \ell) = e^{-r\Delta t} \sum_{j=1}^K P_{kj}(\Delta t) \cdot V(j, \ell + 1) \quad (\text{RS.23})$$

where $P(\Delta t) = \exp(Q \Delta t)$ is the CTMC transition matrix.

2. Immediate exercise value:

$$E(k, \ell) = \max(K - x_n, 0) \quad (\text{RS.24})$$

3. American option value:

$$V(k, \ell) = \max(E(k, \ell), C(k, \ell)) \quad (\text{RS.25})$$

Final Pricing

Given initial price F_0 and initial regime $R_0 = m_0$ (or use stationary distribution π):

1. Find closest price node: $n_0 = \arg\min_n |x_n - F_0|$
2. Compute state index: $k_0 = (m_0 - 1) \cdot N + n_0$
3. Option price: $V(k_0, 0)$

Alternatively:

$$\text{Price} = e^{-r\Delta t} \sum_{j=1}^K P_{k_0 j}(\Delta t) \cdot V(j, 1) \quad (\text{RS.26})$$

State-Dependent Optimal Exercise Boundaries

For each regime m and time t , find critical price $B_m(t)$ such that:

- Exercise if $F_t \leq B_m(t)$ and $R_t = m$
- Continue if $F_t > B_m(t)$

In discrete CTMC framework:

- For each price node x_n in regime m (states $k_n = (m-1)N + n$)
- Compare $E(k_n, \ell)$ and $C(k_n, \ell)$
- Find first node x_{n^*} where $E > C$
- $B_m(t) \approx x_{n^*}$ (use linear interpolation for better accuracy)

Economic insight: In high-volatility regimes (e.g., crisis), the "option to wait" is more valuable, so $B_2(t) < B_1(t)$ (holders delay exercise).

Approximation Error

In this section, we analyze the **approximation error** that occurs when using a continuous-time Markov chain (CTMC) to approximate a diffusion process X_t .

Motivation

We're trying to approximate a complex continuous process X_t (like stock price movement) via CTMC

This approximation inevitably introduces **errors**, and our goal is to **measure** and **control** these errors.

Error Definition

We define the approximation error as:

$$\epsilon(f) = \sup_t |P_t f - P_t^{(n)} f| \leq t \cdot \sup |\mathcal{L}f - Q^{(n)} f| \quad (5.1)$$

- P_t : Semigroup of the diffusion process (how it evolves in time)
- $P_t^{(n)}$: Semigroup of the approximating CTMC
- $\mathcal{L}$: Generator of the diffusion
- $Q^{(n)}$: Generator of the CTMC
- f : A test function

This inequality follows Ethier and Kurtz (1986) Proposition 2.7

$$\|P_t f - P_t^{(n)} f\| \leq t \cdot \|\mathcal{L}f - Q^{(n)} f\|$$

Plainly speaking:

We're exploring: "How far off is the approximation after time t ? It depends on how far apart the generators are."

Detailed Analysis

In this subsection, we want to present a more rigorous derivation of the local truncation error when approximating the diffusion generator $\mathcal{L}$ by the discrete generator $\mathcal{Q}^{(n)}$ of a continuous-time Markov chain (CTMC).

Even though we can general know that the application of the Talor expansion is the contributing factor of the approximation error and the error term can be expressed in the following general form:

$$o(x_{i+1}^h - x_i^h)w_n(x_i^h) - o(x_{i-1}^h - x_i^h)v_n(x_i^h)$$

we still need to conduct a more detailed analysis because one basis point in the accuracy of the error derivation might bring great benefits to the final estimation and future simulation.

Step 1 — Setup

Recall that the diffusion generator is

$$Lf(x) = \mu(x)f'(x) + \frac{1}{2}\sigma^2(x)f''(x),$$

while the CTMC generator $\mathcal{Q}^{(n)}$ acts on the discrete grid points x_i^h as

$$(\mathcal{Q}^{(n)}f)(x_i^h) = f(x_{i+1}^h)q_{i,i+1} + f(x_{i-1}^h)q_{i,i-1} + f(x_i^h)q_{ii}.$$

We want to measure the discrepancy:

$$|Lf - \mathcal{Q}^{(n)}f| = \sum_i \|Lf(x_i^h) - (\mathcal{Q}^{(n)}f)(x_i^h)\|$$

Step 2 — Taylor Expansion

We expand $f(x_{i+1}^h)$ and $f(x_{i-1}^h)$ around x_i^h using Taylor's theorem:

For the forward neighbor:

$$f(x_{i+1}^h) = f(x_i^h) + f'(x_i^h)(x_{i+1}^h - x_i^h) + \frac{1}{2!}f''(x_i^h)(x_{i+1}^h - x_i^h)^2 + \frac{1}{3!}f'''(x_i^h)(x_{i+1}^h - x_i^h)^3 + o((x_{i+1}^h - x_i^h)^3)$$

For the backward neighbor:

$$f(x_{i-1}^h) = f(x_i^h) - f'(x_i^h)(x_i^h - x_{i-1}^h) + \frac{1}{2!}f''(x_i^h)(x_i^h - x_{i-1}^h)^2 - \frac{1}{3!}f'''(x_i^h)(x_i^h - x_{i-1}^h)^3 + o((x_i^h - x_{i-1}^h)^3)$$

Step 3 — Matching Terms

The jump rates $q_{i,i+1} = w_n(x_i^h)$ and $q_{i,i-1} = v_n(x_i^h)$ are chosen so that the first and second derivative terms match those of the diffusion generator $\mathcal{L}$.

After cancellation of the matched terms, only the higher-order remainder terms remain:

$$\sum_i \left\| \frac{1}{3!}f'''(x_i^h) [(x_{i+1}^h - x_i^h)^2 w_n(x_i^h) - (x_i^h - x_{i-1}^h)^2 v_n(x_i^h)] + o((x_{i+1}^h - x_i^h)^2)w_n(x_i^h) - o((x_i^h - x_{i-1}^h)^2)v_n(x_i^h) \right\|$$

Step 4 — Compact Notation

Define

$$h(n, x_i^h) := (x_{i+1}^h - x_i^h)^2 w_n(x_i^h) - (x_i^h - x_{i-1}^h)^2 v_n(x_i^h).$$

Then the local error can be written as:

$$\sum_i \left\| \frac{1}{3!} f'''(x_i^h) h(n, x_i^h) + o(h_{i+1}^2) \left(\frac{h_{i-1} \mu(x_i^h)}{h_i + h_{i-1}} + \frac{\sigma^2(x_i^h)}{h_i + h_{i-1}} \right) - o(h_{i-1}^2) \left(-\frac{h_i \mu(x_i^h)}{h_i + h_{i-1}} + \frac{\sigma^2(x_i^h)}{h_i + h_{i-1}} \right) \right\|$$

where $h_i := x_{i+1}^h - x_i^h$ is the local grid spacing.

Step 5 — Conclusion on Error Order

From the above, the dominant term in the truncation error is:

$$\frac{1}{3!} f'''(x_i^h) h(n, x_i^h),$$

which is **of order $O(h^2)$** for sufficiently smooth f and regular grids.

Thus,

$$\|Lf - Q^{(n)}f\| = O(h^2),$$

and the approximation converges **quadratically** as the mesh size $h \rightarrow 0$.

Plain Language

- The **first derivative** term is matched by adjusting jump rates so drift matches.
- The **second derivative** term is matched so variance matches.
- What's left is the **third derivative** term (and smaller), which is naturally smaller by a factor of h^2 .
- Think of it like approximating a curve with a parabola: if you get the slope and curvature exactly right, the only mismatch is from the cubic bending, which shows up quadratically small in h .

Appendix

Appendix A: Derivation of the Matrix Exponential Formula $P_t = e^{\{tQ\}}$

We aim to derive the closed-form solution of the transition probability matrix P_t for a time-homogeneous continuous-time Markov chain (CTMC) with generator Q .

- Step 1: Kolmogorov Forward Equation

The transition matrix P_t satisfies the **matrix ordinary differential equation (ODE)** known as the Kolmogorov forward equation:

$$\frac{d}{dt} P_t = P_t Q, \quad P_0 = I$$

This is a **first-order linear ODE** for matrix-valued functions.

- Step 2: Solve the Matrix ODE

This matrix ODE is analogous to the scalar ODE:

$$\frac{dy}{dt} = ay, \quad y(0) = 1$$

whose solution is:

$$y(t) = e^{at}$$

Similarly, for matrices, the natural candidate for the solution is the matrix exponential:

$$P_t = e^{tQ} := \sum_{n=0}^{\infty} \frac{(tQ)^n}{n!}$$

We check whether this function satisfies the ODE given in **Step 1**.

Differentiate term by term:

$$\frac{d}{dt} P_t = \sum_{n=1}^{\infty} \frac{d}{dt} \left(\frac{(tQ)^n}{n!} \right) = \sum_{n=1}^{\infty} \frac{(tQ)^{n-1} Q}{(n-1)!} = \left(\sum_{n=0}^{\infty} \frac{(tQ)^n}{n!} \right) Q = P_t Q$$

And clearly:

$$P_0 = e^{0 \cdot Q} = I$$

Thus, the matrix exponential $P_t = e^{tQ}$ satisfies the ODE and the initial condition.

- Step 3: Uniqueness of Solution

Standard theory of matrix differential equations guarantees that this ODE has a **unique solution** (since the right-hand side $(P_t Q)$ is Lipschitz continuous in (P_t)).

Therefore, we conclude that:

$$P_t = e^{tQ}$$

is the **unique solution** to the Kolmogorov forward equation.

Appendix B: Derivation of the Generator of diffusion process via Moment Matching

To construct a generator Q matrix for a continuous-time Markov chain (CTMC) that approximates a given diffusion process, we adopt the **moment matching method**. The idea is to define the discrete generator so that the **first and second moments** (i.e., drift and diffusion) of the original continuous process are preserved.

Strategy: We will use δ functions ($\delta_{i,i+1}$, $\delta_{i,i-1}$) to independently control the first moment μ , and use η function to independently control the second moment σ^2 .

Let x_i^h denote the grid points in the discrete state space, and define the local spacing as:

$$h_i := x_{i+1}^h - x_i^h, \quad h_{i-1} := x_i^h - x_{i-1}^h$$

We aim to define transition rates $q_{i,i+1}$, $q_{i,i-1}$ that reflect the local behavior of drift $\mu(x_i^h)$ and diffusion $\sigma^2(x_i^h)$.

Step 1: Define auxiliary functions

Inspired by Piccioni formula(developed by Piccioni in 1987, a classical method for Markov chain approximation of diffusion processes that separates the treatment of drift and diffusion components), we consider the forward and backward components $w_n(x_i^h)$ and $v_n(x_i^h)$ defined as:

$$\begin{cases} w_n(x_i^h) = \delta_+(x_i^h) + \eta(x_i^h) \\ v_n(x_i^h) = \delta_-(x_i^h) + \eta(x_i^h) \end{cases}$$

According to the conditions given in the former section, our goal is to solve for δ_+ , δ_- and η such that:

- The first moment is matched:

$$\delta_+(x_i^h) - \delta_-(x_i^h) = \mu(x_i^h) \quad (4.20)$$

- The second moment is matched:

$$(x_{i+1}^h - x_i^h)\eta(x_i^h) + (x_i^h - x_{i-1}^h)\eta(x_i^h) = \sigma^2(x_i^h)$$

which implies:

$$\eta(x_i^h) = \frac{\sigma^2(x_i^h)}{(x_{i+1}^h - x_i^h) + (x_i^h - x_{i-1}^h)} \quad (4.23)$$

Moreover, if we want the second moment always being matched in coarsen grid, we need to find an additional condition:

$$(x_{i+1}^h - x_i^h)\delta_+(x_i^h) - (x_i^h - x_{i-1}^h)\delta_-(x_i^h) = 0$$

The additional condition arises because of the numerical stability on non-uniform grids. When approximating diffusion processes using CTMCs on non-uniform grids, simply matching the first and second moments is insufficient. Without the additional constraint, the functions δ_+ and δ_- can become arbitrarily large in coarse grid regions (where spacing h_i is large), leading to:

- Numerical instability in transition rates $q_{i,i+1}$ and $q_{i,i-1}$
- Poor local approximation of the original diffusion process

From the physical perspective, this represents a **local balance condition**: the weighted net flow should be zero, meaning the rightward flow (weighted by right spacing) should balance the leftward flow (weighted by left spacing).

From the geometric perspective, this constraint ensures that the discretized process maintains **local symmetry properties** of the original diffusion process at each grid point. Without this condition, the numerical solution may exhibit artificial "bias" in coarse grid regions, violating the intrinsic balance of the continuous process.

Therefore, we can get the following intermediate results of the two functions:

$$\begin{cases} w_n(x_i^h) = \delta_+(x_i^h) + \frac{\sigma^2(x_i^h)}{(x_{i+1}^h - x_i^h) + (x_i^h - x_{i-1}^h)} \\ v_n(x_i^h) = \delta_-(x_i^h) + \frac{\sigma^2(x_i^h)}{(x_{i+1}^h - x_i^h) + (x_i^h - x_{i-1}^h)} \end{cases}$$

Step 2: Solve for δ_+ , δ_-

Now plug $\eta(x_i^h)$ into (4.20) and form the following system and :

$$\begin{cases} h_i \delta_+(x_i^h) + h_{i-1} \delta_-(x_i^h) = 0 \\ \delta_+(x_i^h) - \delta_-(x_i^h) = \mu(x_i^h) \end{cases}$$

Solving this system gives:

$$\begin{cases} \delta_+(x_i^h) = \frac{h_{i-1} \mu(x_i^h)}{h_i + h_{i-1}} \\ \delta_-(x_i^h) = -\frac{h_i \mu(x_i^h)}{h_i + h_{i-1}} \end{cases} \quad (4.26)$$

Step 3: Final expressions for transition rates

Substitute everything back to get:

$$\begin{cases} w_n(x_i^h) = \frac{h_{i-1} \mu(x_i^h)}{h_i + h_{i-1}} + \frac{\sigma^2(x_i^h)}{h_i + h_{i-1}} \\ v_n(x_i^h) = -\frac{h_i \mu(x_i^h)}{h_i + h_{i-1}} + \frac{\sigma^2(x_i^h)}{h_i + h_{i-1}} \end{cases} \quad (4.27)$$

Therefore, the generator matrix Q has off-diagonal entries:

$$\begin{cases} q_{i,i+1} = \frac{h_{i-1} \mu(x_i^h) + \sigma^2(x_i^h)}{h_i (h_i + h_{i-1})} \\ q_{i,i-1} = \frac{-h_i \mu(x_i^h) + \sigma^2(x_i^h)}{h_{i-1} (h_i + h_{i-1})} \end{cases} \quad (4.28)$$

And the diagonal term is:

$$q_{i,i} = -(q_{i,i+1} + q_{i,i-1})$$

This is the final result shown above and the complete proof is completed

Appendix C: Understanding Jump Terms in Stochastic Processes

In the world of stochastic processes, we often encounter phenomena that don't change smoothly over time. Sometimes, things jump! The jump term we're focusing on is:

$$\int_{\mathbb{R}} h(X_t, y) N(dt, dy)$$

This might look confusing at first, but let's break it down piece by piece and understand what each component means and why it's so powerful for modeling real-world phenomena.

Before the decomposition, let's use the plain language and the concrete examples to gain a better understanding of this relatively unfamiliar and abstract concept.

Plain Language

Think of a jump term as a mathematical way to capture **sudden, unexpected changes** in a system. Unlike smooth changes that happen gradually, jumps are:

- **Instantaneous:** they happen in an instant
- **Random:** we don't know exactly when they'll occur
- **Variable in size:** some jumps are big, some are small

Real-World Examples

Before diving into the math, let's see where jumps appear in real life:

Stock Markets

- Normal trading causes smooth price changes
- Breaking news causes sudden price jumps (up or down)

Population Biology

- Populations usually grow/decline gradually
- Natural disasters cause sudden population drops
- Migration events cause sudden population increases

Next, let's understand each component of our jump term step by step.

Component 1: The Poisson Random Measure $N(dt, dy)$

The term $N(dt, dy)$ is called a **Poisson random measure**. Think of it as a "jump detector" that tells us:

- **When** jumps occur (the dt part)
- **What size** the jumps are (the dy part)

Now, you might have a question—What Does $N(dt, dy)$ Mean?

Actually, the answer is that $N(dt, dy)$ counts the number of jumps that:

- Occur in a tiny time interval $[t, t+dt]$
- Jump with sizes in the range $[y, y+dy]$

Example

Imagine you're sitting by a pond counting fish jumping out of the water:

- dt = a very short time period (like 1 second)
- dy = a range of jump heights (like "between 10cm and 11cm high")
- $N(dt, dy)$ = the number of fish that jump in that time period with that height range

In most of the time, $N(dt, dy) = 0$ (no jumps), but occasionally $N(dt, dy) = 1$ (one jump), and very rarely $N(dt, dy) > 1$ (multiple jumps).

The Expected Behavior: Intensity Measure

Here's where things get mathematically interesting. The **expected value** of $N(dt, dy)$ is:

$$\mathbb{E}[N(dt, dy)] = \nu(dy) \cdot dt$$

Where $\nu(dy)$ is called the **intensity measure** or **jump intensity**. This tells us:

- **How often** jumps of different sizes are expected to occur
- **The average rate** of jumps per unit time

This measure $\nu(dy)$ is also known as the **Lévy measure**, and it plays a central role in defining the jump part of a stochastic process.

In fact, $\nu(dy)$ acts as the **compensator** of the Poisson random measure $N(dt, dy)$, which counts the number of jumps of size y occurring in a time interval of length dt . The key relationship is:

$$\mathbb{E}[N(dt, dy)] = \nu(dy) dt$$

This means $\nu(dy)$ captures the **average intensity** (per unit time) of jumps with size around y , while $N(dt, dy)$ reflects the actual randomness in each sample path.

When we construct the jump generator Q^Δ , we discretize the integral with respect to $\nu(dy)$, so that the Markov chain mimics both the **frequency** and the **distribution** of real-world jumps.

Example: Stock Market Jumps

If we're modeling stock price jumps:

- $\nu(dy)$ might be higher for small jumps (frequent small news)
- $\nu(dy)$ might be lower for large jumps (rare major events)
- $\mathbb{E}[N(dt, dy)] = \nu(dy) \cdot dt$ gives the expected number of jumps in the time interval with length dt

Compensated Poisson Random Measure

So far, we've discussed the original Poisson random measure $N(dt, dy)$, but for theoretical and computational reasons, we often work with its compensated version. In practice, we often work with the **compensated** version:

$$\tilde{N}(dt, dy) = N(dt, dy) - \nu(dy) \cdot dt$$

This is called the **compensated Poisson random measure**.

Here, you might have a question: Why do we need compensation?

The compensated measure $\tilde{N}(dt, dy)$ has a crucial property:

$$\mathbb{E}[\tilde{N}(dt, dy)] = 0$$

This makes it a **martingale**, which is mathematically much more convenient for:

- **Stochastic calculus:** Integration becomes well-defined
- **Expectation calculations:** We can easily compute expected values
- **Convergence proofs:** The theory becomes cleaner

Physical Interpretation

Think of compensation as "removing the predictable part":

- $N(dt, dy)$ = actual jumps that occur
- $\nu(dy) \cdot dt$ = expected jumps (the "baseline")
- $\tilde{N}(dt, dy)$ = unexpected jumps (the "surprise")

This is analogous to removing the "trend" from a time series, leaving only the fluctuations or noise.

Example: News Impact on Stock Prices

Consider stock price jumps due to news:

- **Without compensation:** $N(dt, dy)$ counts all news-related jumps

- **With compensation:** $\tilde{N}(dt, dy)$ counts only the "unexpected" part of news
- The "expected" part $\nu(dy) \cdot dt$ represents the baseline level of news flow

Connection to Our Jump Term

When we write our jump term as:

$$\int_{\mathbb{R}} h(X_t, y) N(dt, dy)$$

The expectation becomes:

$$\mathbb{E} \left[\int_{\mathbb{R}} h(X_t, y) N(dt, dy) \right] = \int_{\mathbb{R}} h(X_t, y) \nu(dy) \cdot dt$$

But when we use the compensated measure:

$$\mathbb{E} \left[\int_{\mathbb{R}} h(X_t, y) \tilde{N}(dt, dy) \right] = 0$$

This is why in stochastic differential equations, we often see terms like:

$$\int_{\mathbb{R}} h(X_t, y) \tilde{N}(dt, dy)$$

To summarize:

- The **Poisson random measure** $N(dt, dy)$ captures the randomness of jump events.
- The **intensity (Lévy) measure** $\nu(dy)$ describes the average rate and distribution of those jumps.
- The **compensated measure** $\tilde{N}(dt, dy) = N(dt, dy) - \nu(dy) dt$ subtracts out the expected part, leaving only the unpredictable "shock," making it mathematically cleaner and better suited for stochastic analysis.

Component 2: The Jump Size Function $h(X_t, y)$

The function $h(X_t, y)$ determines **how big** each jump is. It depends on:

- X_t : The current state of our process
- y : A random "jump driver" variable

Here, you might have another question—Why Does Jump Size Depend on Current State?

In fact, in many real situations, the size of a jump depends on where you currently are

Example 1 - Stock Prices:

- A stock with value 10 might jump by ± 1 on news
- A stock with value 1000 might jump by ± 50 on the same news
- So $h(X_t, y)$ might be proportional to the current price

Example 2 - Population:

- A population of 100 might lose 10 individuals in a disaster
- A population of 10,000 might lose 1,000 individuals in the same disaster
- The jump size scales with population size

Common Forms of $h(X_t, y)$

1. Additive Jumps

$$h(X_t, y) = y$$

- Jump size doesn't depend on current state
- Example: Adding a fixed number of customers to a queue

2. Multiplicative Jumps

$$h(X_t, y) = X_t \cdot y$$

- Jump size proportional to current state
- Example: Stock returns as percentages

3. Mixed Jumps

$$h(X_t, y) = a \cdot X_t + b \cdot y$$

- Combination of both effects
- Example: Population with both absolute and relative effects

Component 3: The Integration $\int_{\mathbb{R}}$

The integral $\int_{\mathbb{R}}$ sums up all possible jump contributions over all possible jump sizes y .

Since y can take any value in $\mathbb{R}$ (positive or negative, small or large), we need to account for all possibilities. The integration ensures we capture:

- Small frequent jumps
- Large rare jumps
- Both positive and negative jumps

After the single analysis of these three components, let's put them together and get the form of our research focus

$$\int_{\mathbb{R}} h(X_t, y) N(dt, dy)$$

This expression represents: "At time t , sum up all the jumps of different sizes y , where each jump of size y contributes $h(X_t, y)$ to the change in X_t ."

Appendix D: Derivation of the Generator of Jump Diffusion process via Moment Matching

Step 1: Basic Construction

Let the jump size function be defined as $h(x, y) = x \cdot y$, and define the midpoints between grid points as:

$$m(y_i) := \frac{x_i^h + x_{i-1}^h}{2}$$

The generator Q^J is constructed as:

$$Q_{i, i+(1+y_i)}^J := \int_{m(y_{i-1})}^{m(y_i)} \nu(i, dy)$$

with diagonal elements given by conservation:

$$Q_{i,i}^J := - \sum_{j \neq i} Q_{i,j}^J$$

This formula is based on the technique of Mijatovi and Pistorius (2009) and ensures that each off-diagonal entry captures the intensity of jumps from state x_i^h to a neighboring state, as determined by integrating the jump measure ν over an appropriate subinterval.

Step 2: Generalization to Arbitrary Jump Functions

To generalize this construction to any jump amplitude function $h(x, y)$, consider a jump from x_i^h to x_j^h . We assume:

$$x_j^h = x_i^h + h(x_i^h, y)$$

Then we invert this to find the corresponding y :

$$y = h^{-1}(x_i^h, x_j^h - x_i^h)$$

The discrete transition rate is then defined as:

$$Q_{x_i^h, x_j^h}^J := \int_{h^{-1}(x_i^h, x_j^h - x_i^h)}^{h^{-1}(x_i^h, x_{j+1}^h - x_i^h)} \nu(x_i^h, dy)$$

and the full matrix Q^J is completed by setting:

$$\begin{aligned} Q_{i,j}^J &= 0 \quad i = 1/N \\ Q_{i,i}^J &= - \sum_{j \neq i} Q_{i,j}^J \end{aligned}$$

Step 3: Matching Moments via Taylor Expansion (verification for the validity of the construction)

We test the operator Q^J on a smooth test function f , and expand $f(x_j^h)$ around x_i^h :

$$f(x_j^h) = f(x_i^h) + f'(x_i^h)(x_j^h - x_i^h) + \frac{1}{2} f''(x_i^h)(x_j^h - x_i^h)^2 + o((x_j^h - x_i^h)^2)$$

Applying Q^J to f :

$$Q^J f(x_i^h) = \sum_j \left[f(x_j^h) - f(x_i^h) \right] Q_{i,j}^J$$

We obtain:

$$Q^J f(x_i^h) \approx f'(x_i^h) \sum_j (x_j^h - x_i^h) Q_{i,j}^J + \frac{1}{2} f''(x_i^h) \sum_j (x_j^h - x_i^h)^2 Q_{i,j}^J$$

This leads to the following **convergence conditions**, ensuring that the discrete jump process captures the same mean and variance as the original process:

- First moment matching:

$$\lim_{n \rightarrow \infty} \sum_j (x_j^h - x_i^h) Q_{i,j}^J = \int_{\mathbb{R}} h(x_i^h, y) \nu(x_i^h, dy)$$

- Second moment matching:

$$\lim_{n \rightarrow \infty} \sum_j (x_j^h - x_i^h)^2 Q_{i,j}^J = \int_{\mathbb{R}} h^2(x_i^h, y) \nu(x_i^h, dy)$$

These two conditions confirm that the operator Q^J correctly approximates the jump term in the continuous generator $\mathcal{L}$, and the resulting CTMC converges to the true jump-diffusion process.

Ultimately, combined with the form of the pure diffusion process, the transition rate matrix of the jump diffusion process is constructed as follows:

$$Q := Q^C + Q^J = \begin{cases} q_{i,i+1} := \left(\frac{1}{h_i} \mu_+(x_i^h) + \frac{\sigma^2(x_i^h) - (h_{i-1} \mu_-(x_i^h) + h_i \mu_+(x_i^h))}{h_i(h_i + h_{i-1})} \right) \\ q_{i,i-1} := \left(\frac{1}{h_{i-1}} \mu_-(x_i^h) + \frac{\sigma^2(x_i^h) - (h_{i-1} \mu_-(x_i^h) + h_i \mu_+(x_i^h))}{h_{i-1}(h_i + h_{i-1})} \right) \\ q_{i,j} = \int_{h^{-1}(x_i^h, x_j^h - x_i^h)} \nu(x_i^h, dy), \quad \forall j \neq i, i-1, i+1 \\ q_{i,i} := - \left(q_{i,i-1} + q_{i,i+1} + \sum_j q_{i,j} \right) \end{cases}$$